

Indice

Costruzione della REST api	5
10.1 REST API Design	5
1. API Design per la risorsa "Books" (Libri)	5
2. API Design per la risorsa "Authors" (Autori)	6
10.2 Author Create Endpoint	8
1. Filosofia Architetture: Disaccoppiare Presentation e Persistence Layer	8
La classe AuthorDto	9
2. Strato di Mappatura: Configurazione di ModelMapper	9
A. Aggiunta della dipendenza e Configurazione del Bean	9
B. Definizione di un'Interfaccia Mapper Generica	10
C. Implementazione del Mapper per l'Autore	10
3. Strato di Servizio (Service Layer)	10
L'interfaccia AuthorService	11
L'implementazione AuthorServiceImpl	11
4. Il Controller: AuthorController	12
Dettagli del flusso nel Controller	12
5. Verifica e Testing con Postman	13
10.3 Test Author Create Endpoint	14
1. Creazione e Setup della Classe di Test	14
Spiegazione tecnica delle annotazioni	14
2. Iniezione delle Dipendenze nel Test	15
3. Primo Test: Verifica dello Status Code 201 Created	16
Ciclo di Refactoring TDD (Red-Green-Refactor)	16
4. Secondo Test: Verifica dei Dati di Risposta tramite JsonPath	16
Dettagli del test	17
10.4 Book Create Endpoint	18
1. Scelta del Metodo HTTP: Perché PUT e non POST?	18
2. Creazione del DTO: BookDto	18
3. Implementazione del Mappatore: BookMapper	19
4. Il Service Layer: BookService e BookServiceImpl	19
L'interfaccia BookService	19
L'implementazione BookServiceImpl	20
5. Il Controller: BookController	21
6. Test Manuale su Postman	22
1. Creazione e Setup della Classe di Test	22
3. Primo Test: Verifica dello Status Code 201 Created	23
4. Secondo Test: Verifica dei Dati di Risposta tramite JsonPath	23
10.5 Author List Endpoint	24
1. Definizione dell'Endpoint nel Controller (AuthorController)	24
2. Implementazione nel Service Layer (AuthorService)	25
3. Verifica con Postman	25
4. Test di Integrazione Automatizzati con MockMvc	26
A. Test di verifica dello status HTTP 200 OK	26

B. Test di verifica del contenuto del Body JSON via JsonPath.....	26
10.6 Book List Endpoint.....	28
1. Definizione dell'Endpoint nel Controller (BookController).....	28
2. Implementazione nel Service Layer (BookService).....	29
L'interfaccia BookService:.....	29
L'implementazione BookServiceImpl:.....	29
3. Verifica Manuale con Postman.....	29
4. Scrittura dei Test di Integrazione Automatizzati.....	30
A. Test di verifica dello status code 200 OK.....	30
B. Test di verifica del contenuto del Body JSON via JsonPath.....	30
10.7 Author Read One Endpoint.....	32
1. Definizione dell'Endpoint nel Controller (AuthorController).....	32
Gestione Reattiva con gli Optional.....	32
2. Implementazione nel Service Layer e Repository.....	33
L'interfaccia AuthorService:.....	33
L'implementazione AuthorServiceImpl:.....	33
3. Verifica Manuale con Postman.....	33
4. Test di Integrazione Automatizzati con MockMvc.....	33
A. Test di verifica dello status 404 Not Found.....	33
B. Test di verifica dello status 200 OK.....	34
C. Test di convalida del payload JSON via JsonPath.....	35
10.8 Book Read One Endpoint.....	36
1. Definizione dell'Endpoint nel Controller (BookController).....	36
2. Implementazione nel Service Layer (BookService).....	37
L'interfaccia BookService:.....	37
L'implementazione BookServiceImpl:.....	37
3. Verifica Manuale con Postman.....	37
4. Test di Integrazione Automatizzati con MockMvc.....	38
A. Test di verifica dello status 200 OK.....	38
B. Test di verifica dello status 404 Not Found.....	38
10.9 Author Full Update Endpoint.....	40
1. Definizione dell'Endpoint nel Controller (AuthorController).....	40
Flusso di Controllo e Logica nel Controller:.....	40
2. Strato di Servizio (AuthorService) e Refactoring di Design.....	41
A. Introduzione di isExists().....	41
B. Refactoring da createAuthor a save.....	42
3. Test Manuale con Postman.....	42
4. Test di Integrazione Automatizzati con MockMvc.....	43
A. Verifica dello Status 404 Not Found per risorsa inesistente.....	43
B. Verifica dello Status 200 OK per risorsa esistente.....	43
C. Verifica dell'effettivo aggiornamento dei dati nel Body JSON.....	44
10.10 Book Full Update Endpoint.....	45
1. Filosofia di Progettazione: PUT come "Create o Update".....	45
2. Modifiche al Service Layer (BookService).....	46

A. Definizione del metodo isExists nell'interfaccia BookService.....	46
B. Implementazione in BookServiceImpl.....	46
C. Rinominazione del Metodo di Salvataggio.....	46
3. Implementazione nel Controller (BookController).....	46
4. Verifica Manuale con Postman.....	47
5. Test di Integrazione Automatizzati con MockMvc.....	47
A. Test di verifica dello status 200 OK.....	48
B. Test di verifica dell'aggiornamento effettivo nel Body JSON.....	49
10.11 Author Partial Update Endpoint.....	50
1. Definizione dell'Endpoint nel Controller (AuthorController).....	50
Flusso logico nel Controller:.....	50
2. Implementazione nel Service Layer (AuthService).....	51
A. Definizione del metodo nell'interfaccia AuthService.....	51
B. Implementazione in AuthServiceImpl.....	51
3. Verifica Manuale con Postman.....	52
4. Test di Integrazione Automatizzati con MockMvc.....	53
A. Test di verifica dello status HTTP 200 OK.....	53
B. Test di verifica della modifica parziale dei dati nel Body JSON.....	54
10.12 Book Partial Update Endpoint.....	55
1. Il Controller: BookController.....	55
2. Lo Strato di Servizio: BookServiceImpl.....	56
A. Definizione nell'interfaccia BookService.....	56
B. Implementazione in BookServiceImpl.....	56
3. Verifica Manuale con Postman.....	57
4. Test di Integrazione Automatizzati con MockMvc.....	57
A. Test di verifica dello status HTTP 200 OK.....	58
B. Test di verifica dell'aggiornamento effettivo via JsonPath.....	59
10.13 Author Delete Endpoint.....	60
1. Definizione dell'Endpoint nel Controller (AuthorController).....	60
2. Implementazione nel Service Layer e Repository.....	61
A. Definizione del metodo nell'interfaccia AuthService.....	61
B. Implementazione in AuthServiceImpl.....	61
3. Verifica Manuale con Postman.....	61
4. Test di Integrazione Automatizzati con MockMvc.....	62
A. Test 1: Eliminazione di un autore non esistente.....	62
B. Test 2: Eliminazione di un autore realmente esistente.....	62
10.14 Book Delete Endpoint.....	64
1. Definizione dell'Endpoint nel Controller (BookController).....	64
2. Implementazione nel Service Layer e Repository.....	65
A. Definizione del metodo nell'interfaccia BookService.....	65
B. Implementazione in BookServiceImpl.....	65
3. Verifica Manuale con Postman.....	65
4. Test di Integrazione Automatizzati con MockMvc.....	66
A. Test 1: Eliminazione di un libro non esistente.....	66

B. Test 2: Eliminazione di un libro realmente esistente.....	67
10.15 Gestione degli Oggetti Nidificati.....	68
1. Il problema: Autore impostato a null.....	68
2. La soluzione: Configurazione della Matching Strategy LOOSE.....	68
Codice di configurazione modificato in MapperConfig:.....	68
3. Come agisce l'unione tra ModelMapper e Spring Data JPA.....	69
4. Verifica finale dell'endpoint.....	70
10.16 Paginazione.....	71
1. Modifiche al Repository Layer.....	71
2. Modifiche al Service Layer.....	71
A. Definizione del metodo nell'interfaccia BookService.....	71
B. Implementazione in BookServiceImpl.....	72
3. Modifiche al Controller Layer (BookController).....	72
4. Ispezione della Risposta JSON in Postman.....	72
5. Controllo della Paginazione tramite Query Parameters.....	73
Deployment su AWS LightSail.....	74
1. Creazione del Dockerfile.....	74
Spiegazione tecnica delle istruzioni:.....	74
2. Generazione del file .jar e Build dell'immagine Docker.....	74
3. Setup del Servizio Container su AWS LightSail.....	75
4. Configurazione del Database PostgreSQL su AWS.....	78
5. Configurazione del Deployment e Mapping delle Variabili d'Ambiente.....	81
6. Gestione dei Problemi di Health Check e Troubleshooting.....	84
7. Test Funzionale dell'API su AWS tramite Postman.....	85

Costruzione della REST api

In questo documento proseguiamo con la costruzione della REST api, quindi sarà utile tutta la teoria citata nel documento precedente.

10.1 REST API Design

La progettazione di un'API REST si basa interamente sul concetto di **risorse**. La struttura stessa degli URL deve riflettere queste risorse. In questa applicazione, le risorse principali sono due: i **libri** (**books**) e gli **autori** di tali libri (**authors**).

Questi dati verranno persistiti all'interno di un database PostgreSQL utilizzando lo strato di persistenza già configurato con Spring Data JPA. Per completare l'architettura REST, occorre quindi implementare un **Service Layer** (strato di servizio) e un **Presentation Layer** (strato di presentazione tramite i controller).

La convenzione fondamentale di REST prevede che l'URL utilizzi sempre il **plurale** per identificare le risorse (quindi **/books** e **/authors**), mai il singolare.

1. API Design per la risorsa "Books" (Libri)

La particolarità introdotta nella gestione dei libri risiede nella gestione degli ID: si è stabilito di **non far generare l'ID automaticamente** dal database, ma di fornirlo manualmente ad ogni inserimento. Questo identificatore unico è il codice **ISBN** del libro.

Poiché l'ID (ISBN) viene fornito direttamente dal client all'atto della creazione, la convenzione REST prevede l'utilizzo del metodo HTTP **PUT** per la creazione, anziché POST (che si usa quando è il server a generare l'ID).

Le operazioni CRUD pianificate per i libri sono così strutturate:

- **Creazione (Create):**
 - **Metodo HTTP:** **PUT**
 - **Endpoint:** **/books/{isbn}**
 - **Codice di stato di successo:** **201 Created**
- **Lettura Singola (Read One):**
 - **Metodo HTTP:** **GET**
 - **Endpoint:** **/books/{isbn}**
 - **Risposta:** Se il libro esiste nel database, restituisce l'oggetto in formato **JSON** e un codice **200 OK**. Se il libro non esiste, restituisce un **corpo di risposta vuoto** e un codice **404 Not Found**.

- **Lettura Multipla (Read Many):**
 - **Metodo HTTP:** GET
 - **Endpoint:** /books (senza specificare l'ISBN)
 - **Risposta:** Restituisce una **lista di libri** e il codice **200 OK**. Questo codice di stato viene restituito **sempre**, anche se il database è vuoto e la lista restituita risulta priva di elementi.
 - **Aggiornamento Completo (Full Update):**
 - **Metodo HTTP:** PUT
 - **Endpoint:** /books/{isbn} (l'URL è identico a quello di creazione)
 - **Risposta:** Restituisce un codice **200 OK** (anziché 201). Questa operazione richiede l'invio di tutti gli attributi dell'oggetto nel corpo della richiesta JSON, andando a sostituire integralmente il record sul database.
 - **Aggiornamento Parziale (Partial Update):**
 - **Metodo HTTP:** PATCH
 - **Endpoint:** /books/{isbn}
 - **Risposta:** Restituisce un codice **200 OK**. A differenza di PUT, con PATCH non è necessario inviare l'intero oggetto; verranno aggiornati sul database esclusivamente i campi forniti nel payload JSON (ad esempio, se si invia solo il campo **title**, solo il titolo verrà modificato).
 - **Eliminazione (Delete):**
 - **Metodo HTTP:** DELETE
 - **Endpoint:** /books/{isbn}
 - **Risposta:** Elimina il record e restituisce **nessun corpo di risposta** con il codice **204 No Content**.
-

2. API Design per la risorsa "Authors" (Autori)

A differenza dei libri, per gli autori si delega la generazione degli identificativi unici (ID numerici) direttamente al sistema (database tramite sequenze).

Di conseguenza, la convenzione architetturale impone l'uso del metodo HTTP **POST** per la creazione.

Le operazioni CRUD pianificate per gli autori sono così strutturate:

- **Creazione (Create):**
 - **Metodo HTTP:** POST
 - **Endpoint:** /authors (senza ID nell'URL)
 - **Risposta:** Restituisce un codice **201 Created** e l'oggetto autore appena generato all'interno del corpo della risposta.

- **Lettura Singola (Read One):**
 - **Metodo HTTP:** GET
 - **Endpoint:** /authors/{id}
 - **Risposta:** Se l'autore viene trovato, restituisce l'oggetto con codice **200 OK**; se non esiste, restituisce il codice **404 Not Found**.
- **Lettura Multipla (Read Many):**
 - **Metodo HTTP:** GET
 - **Endpoint:** /authors
 - **Risposta:** Restituisce una **lista di autori** con codice **200 OK**, indipendentemente dal fatto che ci siano o meno record memorizzati nel database.
- **Aggiornamento Completo (Full Update):**
 - **Metodo HTTP:** PUT
 - **Endpoint:** /authors/{id}
 - **Risposta:** Restituisce il codice **200 OK** e sostituisce integralmente i dati dell'autore presente nel database con l'oggetto fornito nel corpo della richiesta.
- **Aggiornamento Parziale (Partial Update):**
 - **Metodo HTTP:** PATCH
 - **Endpoint:** /authors/{id}
 - **Risposta:** Restituisce il codice **200 OK**. Aggiorna solo gli attributi specifici inclusi nella richiesta (ad esempio, inviando solo il campo **name**, solo il nome dell'autore verrà modificato nel database).
- **Eliminazione (Delete):**
 - **Metodo HTTP:** DELETE
 - **Endpoint:** /authors/{id}
 - **Risposta:** Rimuove l'autore e restituisce un codice **204 No Content** senza alcun corpo di risposta.
 - *Nota sulla gestione delle risorse inesistenti:* Se l'autore non esiste già nel database, l'operazione di cancellazione viene comunque considerata completata con successo (poiché l'obiettivo finale di non avere quella risorsa nel sistema è soddisfatto) e l'endpoint restituisce comunque un codice **204 No Content**.

Questo design fornisce una struttura REST pulita e scalabile per l'applicazione, definendo chiaramente i contratti di comunicazione tra client e server.

10.2 Author Create Endpoint

In questa sezione verrà illustrata l'implementazione dell'endpoint per la **creazione di un autore** (**Author**). Questo passaggio introduce concetti architetturali fondamentali per le applicazioni enterprise, come il disaccoppiamento dei livelli tramite i DTO (Data Transfer Objects) e l'uso di mappatori automatici.

1. Filosofia Architetturale: Disaccoppiare Presentation e Persistence Layer

Prima di tutto in questa sezione bisogna accertarsi che sia presente all'interno del `pom.xml` la dipendenza:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Oltre alla solita:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Fino a questo momento, l'applicazione ha utilizzato gli stessi oggetti Java sia per la persistenza sul database che per la logica di business. Tuttavia, esporre direttamente le entità del database al client tramite i controller è una pratica sconsigliata:

- **Problema:** Un'entità JPA (come **AuthorEntity**) contiene metadati specifici del database (annotazioni come **@Entity**, **@Id**, generatori di sequenze) che il Presentation Layer non dovrebbe conoscere. Se la struttura del database cambia, l'API esterna rischierebbe di rompersi.
- **Soluzione:** Introdurre un **DTO (Data Transfer Object)**, ovvero un oggetto Java semplice (POJO) dedicato esclusivamente al trasferimento dei dati da e verso il client.

Per garantire la netta separazione tra i livelli, si effettua un refactoring architetturale:

1. Rinominare la classe di dominio **Author** in **AuthorEntity** e spostarla nel pacchetto dedicato **entities**.
2. Creare la classe **AuthorDto** all'interno del pacchetto **dto**.

La classe `AuthorDto`

La classe DTO contiene solo i campi necessari alla comunicazione, senza alcuna annotazione JPA:

```
@Data
@AllArgsConstructor
@NoArgsConstructor
@Builder
public class AuthorDto {

    private Long id;

    private String name;

    private Integer age;
}
```

- **Nota su Jackson:** È fondamentale includere l'annotazione `@NoArgsConstructor` di Lombok, poiché Jackson utilizza il costruttore vuoto (senza argomenti) per istanziare l'oggetto prima di popolare i campi tramite i setter in fase di deserializzazione.
-

2. Strato di Mappatura: Configurazione di `ModelMapper`

Per convertire gli oggetti da `AuthorDto` a `AuthorEntity` (e viceversa) senza scrivere codice di conversione manuale ripetitivo, viene introdotta la libreria **ModelMapper**.

A. Aggiunta della dipendenza e Configurazione del Bean

Dopo aver aggiunto la dipendenza nel `pom.xml`:

```
<dependency>
  <groupId>org.modelmapper</groupId>
  <artifactId>modelmapper</artifactId>
  <version>3.0.0</version>
</dependency>
```

Si definisce una classe di configurazione per registrare `ModelMapper` nel contesto di Spring:

```
@Configuration
public class MapperConfig {

    @Bean
    public ModelMapper modelMapper() {
        return new ModelMapper();
    }
}
```

B. Definizione di un'Interfaccia Mapper Generica

Per consentire la futura sostituzione della libreria di mappatura senza impattare il resto dell'applicazione, viene creata un'interfaccia generica chiamata `Mapper<A, B>` nel pacchetto `mappers`:

```
public interface Mapper<A,B> {  
  
    B mapTo(A a);  
    A mapFrom(B b);  
}
```

C. Implementazione del Mapper per l'Autore

La classe concreta `AuthorMapperImpl` implementa questa interfaccia e viene annotata con `@Component` per essere registrata come Bean:

```
@Component  
public class AuthorMapperImpl implements Mapper<AuthorEntity, AuthorDto> {  
  
    private ModelMapper modelMapper;  
  
    public AuthorMapperImpl(ModelMapper modelMapper) {  
        this.modelMapper = modelMapper;  
    }  
  
    @Override  
    public AuthorDto mapTo(AuthorEntity authorEntity) {  
        return modelMapper.map(authorEntity, AuthorDto.class);  
    }  
  
    @Override  
    public AuthorEntity mapFrom(AuthorDto authorDto) {  
        return modelMapper.map(authorDto, AuthorEntity.class);  
    }  
}
```

3. Strato di Servizio (Service Layer)

Il Service Layer funge da ponte tra il Controller e il Repository. Viene creata l'interfaccia `AuthService` e la sua implementazione concreta `AuthServiceImpl` (annotata con `@Service`).

L'interfaccia **AuthService**:

```
public interface AuthService {  
    AuthorEntity save(AuthorEntity authorEntity);  
}
```

L'implementazione **AuthServiceImpl**:

La classe riceve il **AuthorRepository** tramite Constructor Injection ed esegue una chiamata "pass-through" al metodo **save** del repository JPA:

```
@Service  
public class AuthServiceImpl implements AuthService {  
  
    private AuthorRepository authorRepository;  
  
    public AuthServiceImpl(AuthorRepository authorRepository) {  
        this.authorRepository = authorRepository;  
    }  
  
    @Override  
    public AuthorEntity save(AuthorEntity authorEntity) {  
        return authorRepository.save(authorEntity);  
    }  
}
```

4. Il Controller: **AuthorController**

Infine, viene implementato l'endpoint di presentazione. La classe viene annotata con **@RestController**. Al suo interno vengono iniettati sia il servizio che il mappatore tramite Constructor Injection.

```
@RestController
public class AuthorController {

    private AuthorService authorService;

    private Mapper<AuthorEntity, AuthorDto> authorMapper;

    public AuthorController(AuthorService authorService,
Mapper<AuthorEntity, AuthorDto> authorMapper) {
        this.authorService = authorService;
        this.authorMapper = authorMapper;
    }

    @PostMapping(path = "/authors")
    public ResponseEntity<AuthorDto> createAuthor(@RequestBody AuthorDto
author) {
        // Si converte il DTO ricevuto in Entità
        AuthorEntity authorEntity = authorMapper.mapFrom(author);
        // Si salva l'entità sul database tramite il Service
        AuthorEntity savedAuthorEntity = authorService.save(authorEntity);
        // Infine si restituisce il DTO con lo status HTTP 201 Created
        return new ResponseEntity<>(authorMapper.mapTo(savedAuthorEntity),
HttpStatus.CREATED);
        // Con authorMapper.mapTo(savedAuthorEntity) si riconverte l'entità
        salvata in DTO per la risposta
    }
}
```

Dettagli del flusso nel Controller:

1. **@PostMapping(path = "/authors")**: Associa le richieste HTTP POST su **/authors** a questo metodo.
 2. **@RequestBody**: Istruisce Spring a leggere il corpo della richiesta JSON e a usare Jackson per deserializzarlo in un oggetto **AuthorDto**.
 3. **Mappatura**: Il controller non passa mai il DTO direttamente al servizio; lo converte prima in **AuthorEntity**. Allo stesso modo, riconverte il risultato del servizio in **AuthorDto** prima di rispondere al client.
 4. **ResponseEntity con HttpStatus.CREATED (201)**: Inizialmente l'endpoint restituiva semplicemente l'oggetto (rispondendo con un generico **200 OK**). Viene successivamente modificato per avvolgere il DTO in una **ResponseEntity** in modo da restituire correttamente lo status **201 Created**, che rappresenta lo standard RESTful per la creazione di una nuova risorsa.
-

5. Verifica e Testing con Postman

La verifica funzionale dell'endpoint viene eseguita inviando una richiesta HTTP POST tramite Postman all'indirizzo <http://localhost:8080/authors>:

Payload inviato (Body JSON):

```
{
  "name": "Arya Montgomery",
  "age": 80
}
```

- *Nota:* Non viene passato il campo `id` nel JSON, poiché l'ID viene autogenerato dal database PostgreSQL tramite la sequenza associata all'entità.

Risposta ricevuta: Il server restituisce lo status **200 OK** (che non è necessariamente ciò che vogliamo dato che l'ottimale sarebbe HTTP 201 Created ma più avanti aggiusteremo questo problema) e un corpo JSON contenente l'ID generato automaticamente:

```
{
  "id": 1,
  "name": "Arya Montgomery",
  "age": 80
}
```

10.3 Test Author Create Endpoint

Qui affronteremo l'implementazione dei **test di integrazione per l'endpoint di creazione dell'autore**. L'obiettivo è verificare che l'endpoint REST appena creato non solo funzioni correttamente con un database reale (o emulato), ma restituisca anche gli status code e la struttura dati JSON corretti, senza dover avviare manualmente l'applicazione o usare strumenti esterni come Postman.

1. Creazione e Setup della Classe di Test

I test per i controller vengono posizionati nel Presentation Layer dei test. Viene creato un nuovo pacchetto chiamato **controllers** all'interno della cartella dei test (sotto lo stesso gruppo/package dell'applicazione principale) e, al suo interno, la classe **AuthorControllerIntegrationTests**.

La classe necessita di quattro annotazioni fondamentali per configurare l'ambiente di test integrato:

```
@SpringBootTest
@ExtendWith(SpringExtension.class)
@DirtiesContext(classMode=DirtiesContext.ClassMode.AFTER_EACH_TEST_METHOD)
@AutoConfigureMockMvc
public class AuthorControllerIntegrationTests {
    ...
}
```

Spiegazione tecnica delle annotazioni:

- **@SpringBootTest**: Avvia un contesto applicativo di Spring Boot completo (una versione di test dell'applicazione) in modo da poter eseguire i test in condizioni reali.
 - **@ExtendWith(SpringExtension.class)**: Integra il costrutto di JUnit 5 con le funzionalità di test del framework Spring.
 - **@DirtiesContext(...)** con **AFTER_EACH_TEST_METHOD**: Questa configurazione è di vitale importanza. Dice a Spring di ripulire e rigenerare completamente il contesto applicativo (incluso il database H2 in memoria) dopo l'esecuzione di ogni singolo metodo di test. In questo modo si evita la **test pollution** (l'inquinamento dei test), ovvero il rischio che i dati salvati da un test precedente provochino errori (come vincoli di chiave primaria duplicata) nei test successivi.
 - **@AutoConfigureMockMvc**: Configura automaticamente e inserisce nel contesto applicativo un'istanza di **MockMvc**, lo strumento principale che useremo per simulare le chiamate HTTP.
-

2. Iniezione delle Dipendenze nel Test

All'interno della classe di test, vengono dichiarate le variabili d'istanza per **MockMvc** e per **ObjectMapper** (la classe di Jackson per convertire gli oggetti in JSON e viceversa).

A differenza del codice di produzione, nei test di Spring Boot è necessario dichiarare esplicitamente l'annotazione **@Autowired** sul costruttore per abilitare l'iniezione automatica delle dipendenze da parte del framework:

```
private AuthService authService;

private MockMvc mockMvc;

private ObjectMapper objectMapper;

@Autowired
public AuthorControllerIntegrationTests(MockMvc mockMvc, AuthService
authService) {
    this.mockMvc = mockMvc;
    this.authService = authService;
    this.objectMapper = new ObjectMapper(); // Istanziamento manuale di
ObjectMapper per il test
}
```

3. Primo Test: Verifica dello Status Code **201 Created**

Il primo test ha l'obiettivo minimale ma fondamentale di verificare che, inviando una richiesta POST con un payload JSON valido, il server risponda con lo status code corretto per una creazione, ovvero **201 Created**.

```
@Test
public void testThatCreateAuthorSuccessfullyReturnsHttp201Created() throws
Exception {
    // 1. Genera l'entità autore di test usando la classe di utilità
    AuthorEntity testAuthorA = TestDataUtil.createTestAuthorEntity();
    testAuthorA.setId(null); // Assicurati che l'ID sia null perché deve
    // essere autogenerato dal DB

    // 2. Converte l'oggetto Java in una stringa JSON tramite Jackson
    String authorJson = objectMapper.writeValueAsString(testAuthorA);

    // 3. Esegue la chiamata simulata tramite MockMvc
    mockMvc.perform(
        MockMvcRequestBuilders.post("/authors")
            .contentType(MediaType.APPLICATION_JSON)
            .content(authorJson)
    ).andExpect(
        // Verifica che lo stato restituito sia 201
        MockMvcResultMatchers.status().isCreated()
    );
}
```

Ciclo di Refactoring TDD (Red-Green-Refactor):

Inizialmente il test **fallisce** (fase *Red*). Questo accade perché l'endpoint nel controller restituisce l'oggetto mappato direttamente, portando Spring a generare implicitamente uno status code **200 OK**.

Per soddisfare le specifiche di test e portarlo a verde (fase *Green*), si modifica il metodo **createAuthor** all'interno di **AuthorController** per avvolgere la risposta in una **ResponseEntity**.

4. Secondo Test: Verifica dei Dati di Risposta tramite **JsonPath**

Non basta verificare che il server risponda con **201 Created**; dobbiamo assicurarci che i dati restituiti nel corpo della risposta JSON corrispondano esattamente a quelli che abbiamo inviato e che l'ID sia stato effettivamente generato.

Per fare questo, viene utilizzato **JsonPath**, una libreria che permette di navigare e fare asserzioni all'interno di una stringa JSON usando espressioni intuitive. La sintassi parte con il carattere **\$** (che rappresenta la radice del documento JSON) seguito dal nome della proprietà da ispezionare.

```

@Test
public void testThatCreateAuthorSuccessfullyReturnsSavedAuthor() throws
Exception {
    AuthorDto testAuthorA = TestDataUtil.createTestAuthorDtoA();
    testAuthorA.setId(null); // ID impostato a null per l'autogenerazione
    String authorJson = objectMapper.writeValueAsString(testAuthorA);

    mockMvc.perform(
        MockMvcRequestBuilders.post("/authors")
            .contentType(MediaType.APPLICATION_JSON)
            .content(authorJson)
    ).andExpect(
        // Verifica che nel JSON restituito sia presente un campo "id"
        // e che sia un valore numerico
        MockMvcResultMatchers.jsonPath("$.id").isNumber()
    ).andExpect(
        // Verifica che il campo "name" corrisponda al valore inserito
        MockMvcResultMatchers.jsonPath("$.name").value("Abigail Rose")
    ).andExpect(
        // Verifica che il campo "age" corrisponda al valore inserito
        MockMvcResultMatchers.jsonPath("$.age").value(80)
    );
}

```

Dettagli del test:

- **`jsonPath("$.id").isNumber()`**: Poiché l'ID viene autogenerato dal database PostgreSQL/H2 tramite sequenze, non possiamo prevedere con assoluta certezza quale numero intero verrà assegnato (anche se nei test puliti solitamente parte da 1). Di conseguenza, ci si limita ad asserire che il campo **id** esista e sia un **valore numerico**.
- **`jsonPath("$.name").value("Abigail Rose")`** e **`jsonPath("$.age").value(80)`**: Verificano la corrispondenza esatta dei campi testuali e numerici rispetto ai dati di partenza dell'autore "Abigail Rose", che ha un'età impostata a 80 anni.
 - In fase di sviluppo del test, è possibile inserire temporaneamente asserzioni deliberatamente errate per analizzare l'output di errore restituito da MockMvc, per poi correggere i valori ed ottenere il superamento definitivo del test.

Questo approccio garantisce che la logica del Controller, della serializzazione/deserializzazione Jackson, dei mappatori DTO e della persistenza del database funzionino in perfetta armonia.

10.4 Book Create Endpoint

L'implementazione dell'endpoint per la **creazione di un libro** (**Book**) rappresenta un passaggio cruciale perché, rispetto all'autore, introduce una diversa scelta di design architetturale (l'uso di **PUT** anziché **POST**) e la gestione di oggetti complessi e DTO nel Presentation Layer.

1. Scelta del Metodo HTTP: Perché **PUT** e non **POST**?

La prima grande differenza rispetto all'endpoint degli autori risiede nella gestione della chiave primaria:

- **Identificatore Naturale:** Per i libri si utilizza l'**ISBN** (un codice testuale univoco) fornito direttamente dal client all'atto della creazione. Non viene delegata alcuna generazione automatica di sequenze al database.
 - **La Regola di REST:** Quando l'ID della risorsa viene stabilito e inviato dal client, la convenzione REST impone l'uso del metodo **PUT** con l'identificatore incluso nel percorso dell'URL (es. `/books/{isbn}`). Il metodo **POST** si riserva invece a risorse i cui ID sono generati dal server (come l'ID incrementale degli autori).
-

2. Creazione del DTO: **BookDto**

Per evitare che lo strato di presentazione (Controller) esponga direttamente l'entità JPA del database, viene creato un oggetto di trasferimento dati (**Data Transfer Object**) nel pacchetto **dto**:

```
@Data
@AllArgsConstructor
@NoArgsConstructor
@Builder
public class BookDto {

    private String isbn;
    private String title;
    private AuthorDto author;
}
```

- **Disaccoppiamento delle Relazioni:** All'interno di **BookDto**, anziché fare riferimento a un'entità **AuthorEntity**, si fa riferimento al rispettivo **AuthorDto**. Inoltre, vengono omesse tutte le annotazioni JPA (come **@Entity**, **@Id** o **@ManyToOne**), lasciando solo annotazioni Lombok e quelle necessarie a Jackson per la deserializzazione (in particolare il costruttore senza argomenti **@NoArgsConstructor**).

3. Implementazione del Mappatore: **BookMapper**

Per convertire in modo bidirezionale **BookEntity** e **BookDto** senza scrivere codice ripetitivo, si implementa l'interfaccia generica **Mapper**:

```
@Component
public class BookMapperImpl implements Mapper<BookEntity, BookDto> {

    private ModelMapper modelMapper;

    public BookMapperImpl(ModelMapper modelMapper) {
        this.modelMapper = modelMapper;
    }

    @Override
    public BookDto mapTo(BookEntity bookEntity) {
        return modelMapper.map(bookEntity, BookDto.class);
    }

    @Override
    public BookEntity mapFrom(BookDto bookDto) {
        return modelMapper.map(bookDto, BookEntity.class);
    }
}
```

- L'annotazione **@Component** registra la classe come Spring Bean, rendendola disponibile per la Dependency Injection.

4. Il Service Layer: **BookService** e **BookServiceImpl**

Viene definito il contratto per la gestione della persistenza dei libri.

L'interfaccia **BookService**:

```
public interface BookService {
    BookEntity createBook(String isbn, BookEntity book);
}
```

L'implementazione **BookServiceImpl**:

```
@Service
public class BookServiceImpl implements BookService {

    private BookRepository bookRepository;

    public BookServiceImpl(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    @Override
    public BookEntity createBook(String isbn, BookEntity book) {
        book.setIsbn(isbn);
        return bookRepository.save(book);
    }
}
```

- **Meccanismo di Protezione e Sovrascrittura (ISBN Override):** Prima di invocare il metodo `save()` del repository, il servizio forza l'ISBN dell'entità richiamando `bookEntity.setIsbn(isbn)` con il valore proveniente dall'URL. Questo previene una falla comune nelle API REST: se un client malintenzionato o distratto invia un ISBN nel corpo JSON della richiesta diverso da quello specificato nell'URL, il sistema ignora il corpo JSON e usa rigorosamente l'ID presente nel percorso dell'URL.
-

5. Il Controller: **BookController**

Il controller riceve le richieste HTTP, coordina la conversione dei DTO e interagisce con lo strato di servizio:

```
@RestController
public class BookController {

    private BookService bookService;
    private Mapper<BookEntity, BookDto> bookMapper;

    public BookController(Mapper<BookEntity, BookDto> bookMapper,
BookService bookService) {
        this.bookMapper = bookMapper;
        this.bookService = bookService;
    }

    @PutMapping(path = "/books/{isbn}")
    public ResponseEntity<BookDto> createBook(@PathVariable String isbn,
@RequestBody BookDto bookDto) {
        // Converte il DTO ricevuto nel Body in Entità JPA
        BookEntity bookEntity = bookMapper.mapFrom(bookDto);
        // Salva l'entità richiamando il servizio
        BookEntity savedBookEntity = bookService.createBook(isbn,
bookEntity);
        // Riconverte l'entità persistita nel DTO di risposta
        BookDto savedUpdatedBookDto = bookMapper.mapTo(savedBookEntity);
        // Restituisce il DTO con lo status HTTP 201 Created
        return new ResponseEntity(savedUpdatedBookDto, HttpStatus.CREATED);
    }
}
```

- **@PutMapping(path = "/books/{isbn}")**: Associa le chiamate HTTP PUT a questo metodo.
 - **@PathVariable("isbn")**: Estrae il parametro direttamente dall'URL per passarlo al servizio.
 - **@RequestBody**: Legge il payload JSON in ingresso e lo deserializza in un oggetto **BookDto**.
 - **ResponseEntity**: Consente di restituire esplicitamente lo status **201 Created** (anziché un generico 200 OK) per conformarsi alle linee guida REST sulla creazione di risorse.
-

6. Test Manuale su Postman

Nel video viene mostrato come validare l'endpoint effettuando una chiamata **PUT** a <http://localhost:8080/books/978-1-2345-6789-0>:

Payload JSON inviato (con associazione temporanea dell'autore impostata a **null**):

```
{
  "isbn": "978-1-2345-6789-0",
  "title": "The Shadow in the Attic",
  "author": null
}
```

- **Risultato del test di override:** Se si invia un ISBN fittizio nel corpo JSON (es. `"isbn": "999"`) mantenendo `"2"` nell'URL del percorso (`/books/2`), la risposta del server conferma che il sistema ha ignorato il valore nel corpo e ha creato la risorsa usando l'ID dell'URL (`"isbn": "2"`).
 - **Risposta del server:** HTTP Status **201 Created**.
-

1. Creazione e Setup della Classe di Test

Come è stato fatto per la classe **AuthorControllerIntegrationTests**, viene aggiunta la classe **BooksControllerIntegrationTests** dove proseguiremo con i test effettuati anche per il primo citato.

Come la precedente, la classe necessita di quattro annotazioni fondamentali per configurare l'ambiente di test integrato:

```
@SpringBootTest
@ExtendWith(SpringExtension.class)
@DirtiesContext(classMode=DirtiesContext.ClassMode.AFTER_EACH_TEST_METHOD)
@AutoConfigureMockMvc
public class BooksControllerIntegrationTests {
    private MockMvc mockMvc;

    private ObjectMapper objectMapper;

    private BookService bookService;

    @Autowired
    public BooksControllerIntegrationTests(MockMvc mockMvc, BookService bookService) {
        this.mockMvc = mockMvc;
        this.bookService = bookService;
        this.objectMapper = new ObjectMapper();
    }
}
```

3. Primo Test: Verifica dello Status Code 201 Created

Il primo test ha l'obiettivo minimale ma fondamentale di verificare che, inviando una richiesta PUT con un payload JSON valido, il server risponda con lo status code corretto per una creazione, ovvero **201 Created**.

```
@Test
public void testThatCreateBookReturnsHttpStatus201Created() throws
Exception {
    BookDto testBookA = TestDataUtil.createTestBookDtoA(null);
    String bookJson = objectMapper.writeValueAsString(testBookA);

    mockMvc.perform(
        MockMvcRequestBuilders.put("/books/978-1-2345-6789-0")
            .contentType(MediaType.APPLICATION_JSON)
            .content(bookJson)
    ).andExpect(
        MockMvcResultMatchers.status().isCreated()
    );
}
```

4. Secondo Test: Verifica dei Dati di Risposta tramite JsonPath

Non basta verificare che il server risponda con **201 Created**; dobbiamo assicurarci che i dati restituiti nel corpo della risposta JSON corrispondano esattamente a quelli che abbiamo inviato e che l'ID sia stato effettivamente generato.

Per fare questo, viene utilizzato **JsonPath**, una libreria che permette di navigare e fare asserzioni all'interno di una stringa JSON usando espressioni intuitive. La sintassi parte con il carattere **\$** (che rappresenta la radice del documento JSON) seguito dal nome della proprietà da ispezionare.

```
@Test
public void testThatCreateBookReturnsCreatedBook() throws Exception {
    BookDto testBookA = TestDataUtil.createTestBookDtoA(null);
    String bookJson = objectMapper.writeValueAsString(testBookA);

    mockMvc.perform(
        MockMvcRequestBuilders.put("/books/978-1-2345-6789-0")
            .contentType(MediaType.APPLICATION_JSON)
            .content(bookJson)
    ).andExpect(
        MockMvcResultMatchers.jsonPath("$.isbn").value("978-1-2345-6789-0")
    ).andExpect(
        MockMvcResultMatchers.jsonPath("$.title").value("The Shadow in
the Attic")
    );
}
```

10.5 Author List Endpoint

1. Definizione dell'Endpoint nel Controller (**AuthorController**)

L'obiettivo dell'endpoint di lettura multipla è consentire al client di recuperare l'elenco completo degli autori memorizzati nel sistema sotto forma di oggetti DTO, garantendo il disaccoppiamento tra il Presentation Layer e le entità del database.

Nel controller, l'endpoint viene configurato seguendo questi passaggi tecnici:

- **Mapping HTTP:** Si utilizza l'annotazione **@GetMapping** associata al percorso **/authors**. L'uso del plurale rispetta le convenzioni di progettazione RESTful per l'identificazione delle risorse collettive.
- **Firma del Metodo:** Il metodo viene denominato **listAuthors()** e restituisce una **List<AuthorDto>**. Non richiede parametri o variabili di percorso (*Path Variables*) poiché l'operazione deve restituire l'intera collezione.
- **Flusso di Conversione (Stream API):**
 1. Viene invocato lo strato di servizio tramite **authorService.findAll()** per ottenere una lista di entità JPA (**AuthorEntity**).
 2. La lista di entità viene trasformata in uno stream Java (**authors.stream()**).
 3. Tramite l'operatore **.map()**, ogni entità viene convertita nel rispettivo DTO richiamando il metodo di mappatura **authorMapper.mapTo()**.
 4. Infine, gli elementi trasformati vengono raccolti in una nuova lista tramite **.collect(Collectors.toList())** e restituiti al client.

```
@GetMapping(path = "/authors")
public List<AuthorDto> listAuthors() {
    List<AuthorEntity> authors = authorService.findAll();
    return authors.stream()
        .map(authorMapper::mapTo)
        .collect(Collectors.toList());
}
```

Nota di Architettura e Prestazioni: Il recupero non limitato di tutti i record da una tabella di grandi dimensioni non è scalabile per database di grandi dimensioni (ad esempio con miliardi di righe), poiché rischierebbe di esaurire la memoria o causare crash del server. Questo problema verrà risolto successivamente introducendo il meccanismo di **paginazione**.

2. Implementazione nel Service Layer (**AuthService**)

L'interfaccia **AuthService** definisce il contratto dichiarando il metodo **findAll()**:

```
List<AuthorEntity> findAll();
```

Nella classe concreta **AuthServiceImpl**, l'implementazione deve fare da tramite con il database. Tuttavia, si riscontra una discrepanza di tipi: il metodo **findAll()** fornito nativamente dall'interfaccia **CrudRepository** di Spring Data JPA restituisce un tipo **Iterable<T>**. Poiché l'architettura dell'applicazione adotta l'uso delle liste nel Presentation Layer, si rende necessario un processo di conversione all'interno dello strato di servizio:

- **Conversione da **Iterable** a **List****: Per effettuare la conversione in modo efficiente, si utilizza la classe di utilità **StreamSupport**.

Sintassi di Conversione:

```
@Override
public List<AuthorEntity> findAll() {
    return StreamSupport.stream(authorRepository
        .findAll()
        .spliterator(),
        false)
        .collect(Collectors.toList());
}
```

- La chiamata riceve lo *spliterator* dell'oggetto **Iterable** restituito dal repository. Il secondo parametro (**false**) specifica che lo stream non deve essere elaborato in parallelo. Infine, gli elementi vengono raccolti e restituiti come lista standard di entità.

3. Verifica con Postman

Nel video viene mostrato il funzionamento dell'endpoint avviando l'applicazione e inviando una richiesta HTTP **GET** all'indirizzo <http://localhost:8080/authors>:

- **Risposta del Server**: HTTP Status **200 OK**.

Payload JSON: Viene restituito un array JSON contenente gli autori memorizzati. Se nel database è presente l'autore creato nel capitolo precedente, la risposta mostrerà l'oggetto strutturato con i relativi campi:

```
[
  {
    "id": 1,
    "name": "Arya Montgomery",
    "age": 80
  }
]
```

4. Test di Integrazione Automatizzati con MockMvc

Per garantire che l'endpoint funzioni in modo stabile e non subisca regressioni, vengono scritti due appositi test di integrazione all'interno di `AuthorControllerIntegrationTests`.

A. Test di verifica dello status HTTP 200 OK

Questo test assicura che una richiesta GET indirizzata a `/authors` risponda sempre con lo stato di successo 200 OK, anche se la lista dovesse risultare vuota.

```
@Test
public void testThatListAuthorsReturnsHttpStatus200() throws Exception {
    mockMvc.perform(
        MockMvcRequestBuilders.get("/authors")
            .contentType(MediaType.APPLICATION_JSON)
    ).andExpect(MockMvcResultMatchers.status().isOk());
}
```

B. Test di verifica del contenuto del Body JSON via JsonPath

Questo test verifica che la lista restituita dal server contenga effettivamente gli autori inseriti.

A causa dell'annotazione `@DirtiesContext(classMode = AfterEachTestMethod)`, il database viene completamente svuotato dopo l'esecuzione di ogni test. Pertanto, per verificare il corpo della risposta, il test deve prima inserire manualmente un autore di riferimento:

1. **Iniezione del Servizio:** Si dichiara la dipendenza `AuthService` nella classe di test e la si inietta tramite il costruttore annotato con `@Autowired`.
2. **Preparazione del Dato:** Si genera un'entità autore di test (es. Abigail Rose) tramite la classe di utilità e la si persiste nel database richiamando direttamente il servizio (`authorService.createAuthor(testAuthor)`).
3. **Asserzione sul JSON:** Utilizzando `JsonPath`, si esamina la struttura dell'array JSON restituito. L'indice `0` viene utilizzato per puntare al primo elemento dell'array radice (`$`):
 - `$.id`: Verifica che l'identificativo esista (viene impostata l'asserzione sull'ID fisso dell'autore salvato).
 - `$.name`: Verifica che corrisponda ad "Abigail Rose".
 - `$.age`: Verifica che corrisponda all'età impostata (80).

```

@Test
public void testThatListAuthorsReturnsListOfAuthors() throws Exception {
    // Inserisce un autore per popolare il DB isolato del test
    AuthorEntity testAuthorEntityA =
    TestDataUtil.createTestAuthorEntityA();
    authService.save(testAuthorEntityA); // Salva direttamente tramite lo
    strato di servizio

    mockMvc.perform(
        MockMvcRequestBuilders.get("/authors")
            .contentType(MediaType.APPLICATION_JSON)
    ).andExpect(
        MockMvcResultMatchers.jsonPath("$.id").isNumber()
    ).andExpect(
        MockMvcResultMatchers.jsonPath("$.name").value("Abigail
    Rose")
    ).andExpect(
        MockMvcResultMatchers.jsonPath("$.age").value(80)
    );
}

```

Eseguendo i test, l'intero modulo di integrazione passa con successo a verde, confermando la corretta comunicazione tra controller, servizio, mapper e database.

10.6 Book List Endpoint

1. Definizione dell'Endpoint nel Controller (**BookController**)

L'obiettivo di questa sezione è implementare l'endpoint di lettura multipla per i libri (**listBooks**), garantendo che il **Presentation Layer** restituisca esclusivamente oggetti di tipo DTO (**BookDto**) anziché esporre direttamente le entità del database (**BookEntity**).

Nel controller dei libri, l'endpoint viene configurato seguendo questi passaggi tecnici:

- **Associazione HTTP:** Si applica l'annotazione **@GetMapping** associandola al percorso **/books**.
- **Firma del Metodo:** Il metodo viene denominato **listBooks()** e restituisce un oggetto **List<BookDto>**. Non sono previste variabili di percorso o parametri di ricerca in questa fase.
- **Flusso di Conversione (Stream API):**
 1. Il controller invoca lo strato di servizio richiamando **bookService.findAll()** per ottenere la lista di tutti gli oggetti **BookEntity** presenti sul database.
 2. La lista di entità viene trasformata in uno stream Java (**books.stream()**).
 3. Attraverso il metodo **.map()**, ogni singola entità viene convertita nel rispettivo DTO richiamando il metodo di mappatura **bookMapper.mapTo()**.
 4. I DTO così mappati vengono raccolti in una nuova lista tramite **.collect(Collectors.toList())** e restituiti al client.

```
@GetMapping(path = "/books")
public List<BookDto> listBooks() {
    List<BookEntity> books = bookService.findAll();
    return books.stream()
        .map(bookMapper::mapTo)
        .collect(Collectors.toList());
}
```

Nota sulle prestazioni: Nel video si evidenzia che un approccio in cui si recuperano tutti i record indistintamente non è ottimizzato per database reali di grandi dimensioni (ad esempio con miliardi di righe), in quanto l'elaborazione di una mole simile di dati potrebbe causare rallentamenti critici o mandare in crash l'applicazione. Questo problema verrà affrontato e risolto nei capitoli successivi introducendo la **paginazione**.

2. Implementazione nel Service Layer (**BookService**)

Per supportare la chiamata del controller, si estende lo strato di servizio definendo il contratto e la sua implementazione concreta:

L'interfaccia **BookService**:

Viene aggiunto il metodo `findAll()` che restituisce una lista di entità:

```
List<BookEntity> findAll();
```

L'implementazione **BookServiceImpl**:

Nello strato di servizio sorge la necessità di raccordarsi con il repository JPA. Tuttavia, il metodo `findAll()` esposto dall'interfaccia **CrudRepository** di Spring Data JPA restituisce per sua natura un tipo **Iterable<T>**. Per mantenere la coerenza dell'architettura che prevede l'uso delle liste, si effettua una conversione esplicita all'interno del servizio:

- **Conversione tramite **StreamSupport****: Si converte l'oggetto **Iterable** in una **List** standard estraendo lo *spliterator* del repository e definendo uno stream non parallelo (parametro `false`).

```
@Override
public List<BookEntity> findAll() {
    return StreamSupport
        .stream(
            bookRepository.findAll().spliterator(),
            false)
        .collect(Collectors.toList());
}
```

3. Verifica Manuale con Postman

Dopo aver riavviato l'applicazione, viene effettuato un test manuale inviando una richiesta HTTP **GET** all'indirizzo `http://localhost:8080/books`:

- **Risposta del Server**: HTTP Status **200 OK**.
- **Payload JSON**: Viene restituito un array JSON contenente tutti i libri precedentemente registrati nel database (ad esempio "The Shadow in the Attic") strutturati secondo il modello DTO. Se il database è vuoto, l'API restituisce un array vuoto (`[]`) mantenendo comunque lo status **200 OK**.

4. Scrittura dei Test di Integrazione Automatizzati

Per validare la stabilità dell'endpoint, vengono aggiunti due test di integrazione all'interno della classe **BookControllerIntegrationTests** utilizzando **MockMvc**.

A. Test di verifica dello status code 200 OK

Questo test assicura che una richiesta **GET** inviata all'endpoint **/books** generi sempre una risposta positiva di successo (status 200).

```
@Test
public void testThatListBooksReturnsHttpStatus200Ok() throws Exception {
    mockMvc.perform(
        MockMvcRequestBuilders.get("/books")
            .contentType(MediaType.APPLICATION_JSON)
    ).andExpect(
        MockMvcResultMatchers.status().isOk()
    );
}
```

B. Test di verifica del contenuto del Body JSON via **JsonPath**

Questo test verifica che l'array JSON restituito dall'API contenga effettivamente gli oggetti salvati sul database. Poiché i test sono configurati con **@DirtyContext** per ripulire il database dopo ogni esecuzione, è necessario che il test stesso inserisca un libro prima di inviare la richiesta GET:

1. **Iniezione di **BookService****: Si dichiara l'istanza del servizio all'interno della classe di test e la si inietta tramite il costruttore annotato con **@Autowired**.
2. **Preparazione dei Dati**: Si genera un'entità libro di test (es. tramite la classe di utilità **TestDataUtil.createTestBookEntityA()**) e la si persiste nel database isolato chiamando direttamente il servizio (**bookService.createBook()**).
3. **Asserzioni sul JSON**: Utilizzando **JsonPath**, si esamina la risposta dell'endpoint. Poiché viene restituito un array, l'ispezione richiede l'indicizzazione **\$** per accedere alle proprietà del primo libro presente nella lista:
 - **\$.isbn**: Verifica che corrisponda al codice ISBN del libro salvato.
 - **\$.title**: Verifica che corrisponda al titolo inserito (ad esempio "The Shadow in the Attic").

```

@Test
public void testThatListBooksReturnsBook() throws Exception {
    // Crea e salva un libro di test nel DB isolato
    BookEntity testBookEntityA = TestDataUtil.createTestBookEntityA(null);
    bookService.createBook(testBookEntityA.getIsbn(), testBookEntityA);

    // Esegue la chiamata GET e asserisce sul primo elemento dell'array
    JSON
    mockMvc.perform(
        MockMvcRequestBuilders.get("/books")
            .contentType(MediaType.APPLICATION_JSON)
    ).andExpect(
        MockMvcResultMatchers.jsonPath("$.isbn").value("978-1-2345-6789-0")
    ).andExpect(
        MockMvcResultMatchers.jsonPath("$.title").value("The Shadow
in the Attic")
    );
}

```

Una volta eseguiti, i test passano a verde, validando formalmente il comportamento del nuovo endpoint di lettura collettiva dei libri.

10.7 Author Read One Endpoint

L'implementazione dell'endpoint per il recupero di un singolo autore (**Author Read One**) rappresenta un passaggio fondamentale per completare il ciclo di lettura dell'API. Questa sezione illustra come gestire la ricerca per ID in modo sicuro ed efficiente, restituendo i corretti codici di stato HTTP e implementando i relativi test di integrazione automatizzati.

1. Definizione dell'Endpoint nel Controller (**AuthorController**)

Per recuperare un singolo autore basandosi sul suo identificativo univoco (ID), l'endpoint nel controller viene configurato seguendo questi passaggi tecnici:

- **Mapping HTTP e Path:** Viene utilizzata l'annotazione **@GetMapping** associata al percorso **/authors/{id}**. Le parentesi graffe **{id}** indicano che l'ID è un parametro dinamico del percorso (*path variable*).
- **Controllo dello Stato HTTP:** Il metodo del controller restituisce un oggetto **ResponseEntity<AuthorDto>**. Questo consente di controllare esplicitamente lo status code HTTP da restituire al client in base all'esito della ricerca (es. **200 OK** se l'autore esiste, **404 Not Found** se non esiste).
- **@PathVariable:** L'annotazione **@PathVariable("id")** inserita nella firma del metodo istruisce Spring Boot ad estrarre il valore dall'URL e a mapparlo sul parametro Java di tipo **Long id**.

```
@GetMapping(path = "/authors/{id}")
public ResponseEntity<AuthorDto> getAuthor(@PathVariable("id") Long id) {
    Optional<AuthorEntity> foundAuthor = authorService.findOne(id);
    return foundAuthor.map(authorEntity -> {
        AuthorDto authorDto = authorMapper.mapTo(authorEntity);
        return new ResponseEntity<>(authorDto, HttpStatus.OK);
    }).orElse(new ResponseEntity<>(HttpStatus.NOT_FOUND));
}
```

Gestione Reattiva con gli **Optional**

Il controller gestisce l'assenza o la presenza della risorsa sfruttando la potenza della classe **Optional**:

1. Viene invocato il Service Layer, il cui metodo restituisce un **Optional<AuthorEntity>**.
 2. Attraverso il metodo **.map()** dell'**Optional**, se l'entità è presente, questa viene convertita in **AuthorDto** mediante il mappatore (**authorMapper.mapTo**) e restituita avvolta in una **ResponseEntity** con status **200 OK**.
 3. Se l'**Optional** risulta vuoto, viene eseguita la clausola **.orElse()**, che restituisce una **ResponseEntity** priva di corpo e con lo status code **404 Not Found**.
-

2. Implementazione nel Service Layer e Repository

Per supportare la chiamata del controller, si definisce la logica di business nel Service Layer:

L'interfaccia **AuthService**:

Viene dichiarato il metodo **findOne** che restituisce un **Optional**:

```
Optional<AuthorEntity> findOne(Long id);
```

L'implementazione **AuthServiceImpl**:

Il metodo effettua una semplice chiamata "pass-through" sfruttando il metodo nativo **findById** fornito dall'interfaccia **CrudRepository** di Spring Data JPA:

```
@Override
public Optional<AuthorEntity> findOne(Long id) {
    return authorRepository.findById(id);
}
```

3. Verifica Manuale con Postman

Nel video viene avviata l'applicazione per testare manualmente il comportamento dell'endpoint:

Caso di Successo: Inviando una richiesta **GET** a **http://localhost:8080/authors/1** (dove l'autore con ID 1 è presente nel DB), il server risponde con **200 OK** e restituisce il corpo JSON dell'autore:

```
{
  "id": 1,
  "name": "Arya Montgomery",
  "age": 80
}
```

- **Caso di Errore:** Inviando una richiesta **GET** a un ID inesistente (es. **http://localhost:8080/authors/99**), il server restituisce correttamente un codice **404 Not Found** e un corpo di risposta vuoto.

4. Test di Integrazione Automatizzati con **MockMvc**

Per assicurare la stabilità dell'endpoint ed evitare regressioni nel tempo, all'interno della classe **AuthorControllerIntegrationTests** vengono aggiunti tre test di integrazione mirati.

A. Test di verifica dello status **404 Not Found**

Questo test assicura che, se si richiede un autore inesistente, l'endpoint risponda con lo status code corretto. Poiché il database viene completamente ripulito dopo ogni test grazie a `@DirtiesContext`, richiedendo un ID casuale (es. 99) siamo sicuri del fallimento della ricerca.

```
@Test
public void testThatGetAuthorReturnsHttpStatus404WhenNoAuthorExists()
throws Exception {
    mockMvc.perform(
        MockMvcRequestBuilders.get("/authors/99")
            .contentType(MediaType.APPLICATION_JSON)
    ).andExpect(MockMvcResultMatchers.status().isNotFound());
}
```

B. Test di verifica dello status 200 OK

Questo test verifica che l'endpoint risponda con `200 OK` se l'autore richiesto è presente nel database. Per fare ciò, il test deve prima salvare un autore di prova nel database isolato richiamando direttamente lo strato di servizio (`authorService.save`).

```
@Test
public void testThatGetAuthorReturnsHttpStatus200WhenAuthorExist() throws
Exception {
    // Salva un autore di test nel database isolato
    AuthorEntity testAuthorEntityA =
TestDataUtil.createTestAuthorEntityA();
    authorService.save(testAuthorEntityA);

    mockMvc.perform(
        MockMvcRequestBuilders.get("/authors/1") //oppure: ("/authors/"
+ testAuthorEntityA.getId())
            .contentType(MediaType.APPLICATION_JSON)
    ).andExpect(MockMvcResultMatchers.status().isOk());
}
```

C. Test di convalida del payload JSON via **JsonPath**

Questo test si spinge oltre la semplice verifica dello status code, assicurando che i dati presenti nel corpo JSON restituito dal server corrispondano esattamente ai dati dell'autore precedentemente salvato.

```
@Test
public void testThatGetAuthorReturnsAuthorWhenAuthorExist() throws
Exception {
    // Salva l'autore nel DB
    AuthorEntity testAuthorEntityA =
TestDataUtil.createTestAuthorEntityA();
    authorService.save(testAuthorEntityA);

    mockMvc.perform(
        MockMvcRequestBuilders.get("/authors/1") //oppure: ("/authors/"
+ testAuthorEntityA.getId())
        .contentType(MediaType.APPLICATION_JSON)
        // Trattandosi di un singolo oggetto e non di una lista, usiamo
direttamente $.id, $.name e $.age
    ).andExpect(
        MockMvcResultMatchers.jsonPath("$.id").value(1) //oppure:
(testAuthorEntityA.getId())
    ).andExpect(
        MockMvcResultMatchers.jsonPath("$.name").value("Abigail Rose")
    ).andExpect(
        MockMvcResultMatchers.jsonPath("$.age").value(80)
    );
}
```

I test così configurati confermano che l'intero flusso di recupero singolo risponde in modo solido e perfettamente conforme ai contratti delle specifiche REST.

10.8 Book Read One Endpoint

L'implementazione dell'endpoint per il recupero di un singolo libro (**Book Read One**) permette di ricercare un libro specifico tramite la sua chiave primaria naturale, ovvero il codice **ISBN**. Come per l'autore, la gestione dell'endpoint si basa sul corretto utilizzo dei codici di stato HTTP (**200 OK** e **404 Not Found**) e sul disaccoppiamento dei dati tramite i DTO.

1. Definizione dell'Endpoint nel Controller (**BookController**)

Nel controller dei libri, l'endpoint viene configurato per mappare le richieste **GET** contenenti l'ISBN nel percorso dell'URL:

- **Mapping HTTP e Path:** Si utilizza l'annotazione **@GetMapping** associata al percorso **/books/{isbn}**. La sezione **{isbn}** rappresenta la variabile dinamica che identifica univocamente la risorsa.
- **Firma del Metodo:** Il metodo **getBook** riceve il parametro dinamico tramite l'annotazione **@PathVariable("isbn")** (mappato come **String isbn**) e restituisce un oggetto **ResponseEntity<BookDto>** per poter controllare lo stato HTTP di risposta.
- **Gestione dell'Optional:** Lo strato di servizio restituisce un **Optional<BookEntity>**. Il controller esegue una mappatura condizionale:
 1. Se il libro è presente, viene convertito in **BookDto** tramite il mappatore (**bookMapper.mapTo()**) e restituito con lo stato **200 OK**.
 2. Se l'oggetto **Optional** è vuoto (libro non trovato), viene restituita una risposta priva di corpo con lo stato **404 Not Found**.

```
@GetMapping(path = "/books/{isbn}")
public ResponseEntity<BookDto> getBook(@PathVariable("isbn") String isbn)
{
    Optional<BookEntity> foundBook = bookService.findOne(isbn);
    return foundBook.map(bookEntity -> {
        BookDto bookDto = bookMapper.mapTo(bookEntity);
        return new ResponseEntity<>(bookDto, HttpStatus.OK);
    }).orElse(new ResponseEntity<>(HttpStatus.NOT_FOUND));
}
```

2. Implementazione nel Service Layer (**BookService**)

La logica di recupero viene definita nel Service Layer per interfacciarsi con il database tramite il repository JPA.

L'interfaccia **BookService**:

Viene dichiarato il metodo **findOne** che accetta l'ISBN come stringa e restituisce un **Optional**:

```
Optional<BookEntity> findOne(String isbn);
```

L'implementazione **BookServiceImpl**:

La classe concreta esegue una chiamata "pass-through" sfruttando il metodo nativo **findById** fornito da **CrudRepository** (poiché per l'entità **BookEntity** la chiave primaria è proprio la stringa ISBN):

```
@Override
public Optional<BookEntity> findOne(String isbn) {
    return bookRepository.findById(isbn);
}
```

3. Verifica Manuale con Postman

Per testare manualmente il corretto comportamento dell'endpoint, si effettua una chiamata a server attivo:

- **Caso di Successo:** Inviando una richiesta **GET** a **http://localhost:8080/books/978-1-2345-6789-0** (con un ISBN esistente nel database), l'API restituisce lo status code **200 OK** e il corpo JSON strutturato del libro (es. "The Shadow in the Attic").
 - **Caso di Errore:** Inviando una richiesta a un ISBN inesistente, il server risponde correttamente con **404 Not Found** e un corpo vuoto.
-

4. Test di Integrazione Automatizzati con MockMvc

All'interno della classe **BookControllerIntegrationTests**, vengono scritti i test di integrazione per verificare automaticamente la stabilità di questo endpoint sia in caso di successo che di assenza del record.

A. Test di verifica dello status 200 OK

Questo test simula il recupero di un libro esistente. Grazie all'isolamento del database garantito da **@DirtiesContext**, il test deve prima creare e persistere un libro di prova prima di effettuarne la richiesta di lettura:

```
@Test
public void testThatGetBookReturnsHttpStatus200OkWhenBookExists() throws
Exception {
    // Crea e salva un libro di test nel DB isolato
    BookEntity testBookEntityA = TestDataUtil.createTestBookEntityA(null);
    bookService.createBook(testBookEntityA.getIsbn(), testBookEntityA);

    // Esegue la chiamata GET e verifica lo status code 200
    mockMvc.perform(
        MockMvcRequestBuilders.get("/books/" +
testBookEntityA.getIsbn())
            .contentType(MediaType.APPLICATION_JSON)
        ).andExpect(
            MockMvcResultMatchers.status().isOk()
        );
}
```

B. Test di verifica dello status 404 Not Found

Questo test assicura che, in caso di ricerca di un ISBN non censito, l'applicazione risponda correttamente con lo stato di errore previsto dalle specifiche REST:

```
@Test
public void testThatGetBookReturnsHttpStatus404WhenBookDoesntExist()
throws Exception {
    BookEntity testBookEntityA = TestDataUtil.createTestBookEntityA(null);
    mockMvc.perform(
        // Si potrebbe anche evitare la creazione di un testBookEntity
e fare il test con un isbn fittizio come:
MockMvcRequestBuilders.get("/books/978-0-0000-0000-0")
        MockMvcRequestBuilders.get("/books/" +
testBookEntityA.getIsbn())
            .contentType(MediaType.APPLICATION_JSON)
        ).andExpect(
            MockMvcResultMatchers.status().isNotFound()
        );
}
```

La corretta esecuzione di questa suite di test garantisce che la logica di gestione delle letture singole per la risorsa **Book** sia robusta ed esente da regressioni.

10.9 Author Full Update Endpoint

In questo capitolo spiegheremo dettagliatamente l'implementazione dell'endpoint per l'aggiornamento completo (Full Update) di un autore. Nello stile RESTful, un aggiornamento completo viene eseguito tramite il metodo HTTP **PUT** e ha l'obiettivo di sostituire integralmente lo stato di una risorsa esistente con i dati forniti nel payload.

1. Definizione dell'Endpoint nel Controller (**AuthorController**)

L'endpoint nel controller viene configurato per gestire le richieste **PUT** indirizzate a uno specifico autore tramite il suo ID.

- **Mapping HTTP:** Si utilizza l'annotazione **@PutMapping** associata al percorso **/authors/{id}**.
- **Firma del Metodo:** Il metodo **fullUpdateAuthor** (o **updateAuthor**) accetta l'ID estratto dall'URL tramite **@PathVariable("id")** e il payload JSON deserializzato in un oggetto DTO tramite **@RequestBody**. Restituisce un oggetto **ResponseEntity<AuthorDto>** per poter controllare lo stato HTTP di risposta.

Flusso di Controllo e Logica nel Controller:

1. **Verifica dell'Esistenza:** Prima di tentare qualsiasi modifica, il controller interroga lo strato di servizio richiamando **authorService.exists(id)**.
2. **Gestione Risorsa Inesistente:** Se l'autore non esiste nel database, REST impone di non creare o aggiornare nulla, ma di restituire immediatamente una risposta vuota con codice di stato **404 Not Found**.
3. **Allineamento dell'ID:** Se l'autore esiste, si forza l'ID dell'oggetto DTO impostandolo con il valore estratto dal percorso dell'URL (**id**), garantendo che i dati siano perfettamente allineati anche se il client ha inviato un ID differente o nullo nel corpo JSON.
4. **Mappatura e Salvataggio:**
 - Il DTO aggiornato viene convertito in un'entità JPA utilizzando **authorMapper.mapFrom(authorDto)**.
 - L'entità viene salvata richiamando il metodo **.save()** dello strato di servizio.
 - L'entità salvata viene nuovamente convertita in un DTO tramite **authorMapper.mapTo(savedAuthorEntity)**.
5. **Risposta di Successo:** Viene restituito il DTO aggiornato all'interno di una **ResponseEntity** con lo status code **200 OK**.

```

@PutMapping(path = "/authors/{id}")
public ResponseEntity<AuthorDto> fullUpdateAuthor(
    @PathVariable("id") Long id,
    @RequestBody AuthorDto authorDto) {

    // Verifica se l'autore esiste
    if(!authorService.exists(id)) {
        return new ResponseEntity<>(HttpStatus.NOT_FOUND); // 404
    }

    // Allinea l'ID del DTO con quello del percorso URL
    authorDto.setId(id);
    // Converte il DTO in Entità
    AuthorEntity authorEntity = authorMapper.mapFrom(authorDto);
    // Salva l'entità aggiornata nel database
    AuthorEntity savedAuthorEntity = authorService.save(authorEntity);
    // Riconverte l'entità in DTO e restituisce 200 OK
    return new ResponseEntity<>(
        authorMapper.mapTo(savedAuthorEntity),
        HttpStatus.OK);
}

```

2. Strato di Servizio (**AuthorService**) e Refactoring di Design

L'introduzione della funzionalità di aggiornamento porta a prendere una decisione di design: possiamo creare un nuovo metodo specifico nel nostro servizio chiamato **fullUpdateAuthor**, oppure possiamo rendere più generico il metodo che abbiamo già, ovvero **createAuthor**.

La scelta ottimale ricade sulla seconda opzione, il perchè è descritto a fine sottosezione per avere uno sguardo completo. Quindi procedendo:

A. Introduzione di **exists()**

Si definisce un metodo per verificare la presenza dell'autore nel database.

Interfaccia (**AuthorService**):

```
boolean exists(Long id);
```

Implementazione (AuthorServiceImpl): Il metodo esegue una chiamata pass-through al repository JPA sfruttando il metodo nativo **existsById(id)** di **CrudRepository**:

```

@Override
public boolean exists(Long id) {
    return authorRepository.existsById(id);
}

```

B. Refactoring da `createAuthor` a `save`

Nel Service Layer, il metodo precedentemente chiamato `createAuthor` viene rinominato in `save`.

- **Perché?** In Spring Data JPA, l'interfaccia `CrudRepository` utilizza lo stesso identico metodo `.save()` sia per inserire un nuovo record (se l'ID è nullo o non presente a DB) sia per aggiornarne uno esistente (se l'ID è già presente a DB). Rinominare il metodo del servizio in `save` rende il codice molto più coerente e riutilizzabile sia per l'endpoint di creazione che per quello di aggiornamento.
-

3. Test Manuale con Postman

Il video mostra come validare l'endpoint appena creato simulando le chiamate HTTP tramite Postman:

- **Scenario 1 (Aggiornamento):** Avendo già un autore nel database (ID: 1, nome: "Arya Montgomery"), si esegue una chiamata **PUT** a `http://localhost:8080/authors/1`. Nel corpo JSON si invia il nome modificato in "Maria Montgomery" e un ID fittizio (es. `"id": 99`). Il server ignora l'ID del corpo, applica l'aggiornamento sulla risorsa 1, e restituisce lo status **200 OK** confermando la modifica a "Maria Montgomery".
 - **Scenario 2 (Risorsa Inesistente):** Inviando una richiesta **PUT** a un ID inesistente (es. `http://localhost:8080/authors/99`), l'applicazione risponde istantaneamente con un codice **404 Not Found**.
-

4. Test di Integrazione Automatizzati con MockMvc

Per consolidare l'endpoint all'interno di `AuthorControllerIntegrationTests`, vengono scritti tre test di integrazione mirati.

A. Verifica dello Status 404 Not Found per risorsa inesistente

Questo test assicura che il tentativo di aggiornare un autore non presente a database fallisca con lo status code corretto.

```
@Test
public void
testThatFullUpdateAuthorReturnsHttpStatus404WhenNoAuthorExists() throws
Exception {
    AuthorDto testAuthorDtoA = TestDataUtil.createTestAuthorDtoA();
    String authorDtoJson = objectMapper.writeValueAsString(testAuthorDtoA);
    // Genera un DTO di test
    mockMvc.perform(
        MockMvcRequestBuilders.put("/authors/99") // ID 99 non esiste
        nel DB di test pulito
            .contentType(MediaType.APPLICATION_JSON)
            .content(authorDtoJson)
    ).andExpect(MockMvcResultMatchers.status().isNotFound()); // Verifica
    il 404
}
```

B. Verifica dello Status 200 OK per risorsa esistente

Questo test assicura che, se la risorsa esiste, l'aggiornamento risponda con lo stato di successo 200 OK.

```
@Test
public void
testThatFullUpdateAuthorReturnsHttpStatus200WhenAuthorExists() throws
Exception {
    // Crea e salva un autore reale nel DB di test
    AuthorEntity testAuthorEntityA =
    TestDataUtil.createTestAuthorEntityA();
    AuthorEntity savedAuthor = authService.save(testAuthorEntityA);

    // Prepara il DTO per la modifica
    AuthorDto testAuthorDtoA = TestDataUtil.createTestAuthorDtoA();
    String authorDtoJson = objectMapper.writeValueAsString(testAuthorDtoA);

    mockMvc.perform(
        MockMvcRequestBuilders.put("/authors/" + savedAuthor.getId())
    // Usa l'ID salvato
        .contentType(MediaType.APPLICATION_JSON)
        .content(authorDtoJson)
    ).andExpect(MockMvcResultMatchers.status().isOk()); // Verifica il 200
}
```

C. Verifica dell'effettivo aggiornamento dei dati nel Body JSON

Questo test verifica che i vecchi dati (Abigail Rose, età 80) vengano effettivamente sostituiti sul database dai dati del nuovo DTO (Thomas Cronin, età 44) e che il server restituisca la nuova rappresentazione corretta mantenendo l'ID originario.

```
@Test
public void testThatFullUpdateUpdatesExistingAuthor() throws Exception {
    // Crea e salva l'autore "Abigail Rose" (ID originario) nel DB di test
    AuthorEntity testAuthorEntityA =
    TestDataUtil.createTestAuthorEntityA();
    AuthorEntity savedAuthor = authService.save(testAuthorEntityA);

    // Prepara il DTO di "Thomas Cronin" per sovrascrivere l'autore
    // esistente
    AuthorEntity authorDto = TestDataUtil.createTestAuthorB();
    authorDto.setId(savedAuthor.getId()); // Allinea l'ID con quello
    // salvato nel database

    String authorDtoUpdateJson =
    objectMapper.writeValueAsString(authorDto);

    mockMvc.perform(
        MockMvcRequestBuilders.put("/authors/" + savedAuthor.getId())
            .contentType(MediaType.APPLICATION_JSON)
            .content(authorDtoUpdateJson)
    ).andExpect(
        // Verifica che l'ID sia rimasto lo stesso
        MockMvcResultMatchers.jsonPath("$.id").value(savedAuthor.getId())
    ).andExpect(
        // Verifica che il nome sia stato aggiornato a "Thomas Cronin"
        MockMvcResultMatchers.jsonPath("$.name").value(authorDto.getName())
    ).andExpect(
        // Verifica che l'età sia stata aggiornata a 44
        MockMvcResultMatchers.jsonPath("$.age").value(authorDto.getAge())
    );
}
```

Questi passaggi completano l'implementazione e il collaudo robusto dell'aggiornamento degli autori in conformità agli standard delle API RESTful.

10.10 Book Full Update Endpoint

In questa sezione viene illustrata la progettazione, l'implementazione e il collaudo dell'endpoint per l'aggiornamento completo (*Full Update*) di un libro. Questa operazione presenta una particolarità architettuale unica rispetto alla risorsa degli autori: l'utilizzo del medesimo endpoint **PUT** sia per l'azione di creazione sia per quella di aggiornamento.

1. Filosofia di Progettazione: **PUT** come "Create o Update"

La scelta di design effettuata per la risorsa **Book** prevede che sia il client a fornire l'identificatore univoco (il codice **ISBN**) al momento della creazione. Di conseguenza, l'endpoint per la creazione utilizza già il metodo HTTP **PUT** associato al percorso `/books/{isbn}`.

Nello standard RESTful, quando si usa **PUT** con un ID fornito dal client, l'endpoint deve essere **idempotente** e in grado di gestire due scenari:

1. **Se la risorsa NON esiste:** Il server la crea da zero e restituisce lo status code **201 Created**.
2. **Se la risorsa ESISTE:** Il server aggiorna interamente lo stato della risorsa esistente con i dati inviati nel payload e restituisce lo status code **200 OK**.

Per questo motivo, l'endpoint esistente nel controller viene convertito e rinominato da "creazione semplice" a un endpoint unificato di "**creazione/aggiornamento**".

2. Modifiche al Service Layer (**BookService**)

Per consentire al controller di distinguere tra creazione e aggiornamento, lo strato di servizio deve implementare un metodo di verifica dell'esistenza.

A. Definizione del metodo **isExists** nell'interfaccia **BookService**

```
boolean isExists(String isbn);
```

B. Implementazione in **BookServiceImpl**

Il metodo effettua una chiamata pass-through al repository JPA sfruttando il metodo nativo **existsById**:

```
@Override
public boolean isExists(String isbn) {
    return bookRepository.existsById(isbn);
}
```

C. Rinominazione del Metodo di Salvataggio

Il metodo precedentemente denominato **createBook** nel servizio viene rinominato in **createUpdateBook** (o mantenuto generico come **save**), per riflettere il fatto che la logica sottostante di Spring Data JPA (il metodo **.save()** del repository) esegue autonomamente un inserimento (*insert*) o una sovrascrittura (*update*) a seconda che l'ID sia già registrato o meno sul database.

3. Implementazione nel Controller (**BookController**)

Il metodo del controller viene modificato per accogliere la logica condizionale dello status code.

- **Rinominazione:** Il metodo viene rinominato in **createUpdateBook**.
- **Flusso di Controllo:**
 1. Si converte il **BookDto** in ingresso in una **BookEntity**.
 2. Si verifica se il libro esiste già richiamando **bookService.isExists(isbn)**.
 3. Si esegue il salvataggio tramite **bookService.createUpdateBook(isbn, bookEntity)**.
 4. Si mappa l'entità risultante nel DTO di risposta.
 5. **Verifica Condizionale:** Se il boolean **bookExists** è vero, si risponde con lo status **200 OK**; se è falso, si risponde con **201 Created**.

```
@PutMapping(path = "/books/{isbn}")
public ResponseEntity<BookDto> createUpdateBook(@PathVariable String isbn,
@RequestBody BookDto bookDto) {
    // Converti il DTO in Entità
    BookEntity bookEntity = bookMapper.mapFrom(bookDto);
    // 2. Verifica se il libro esiste prima di eseguire l'operazione
    boolean bookExists = bookService.exists(isbn);
    // 3. Salva o aggiorna il record
    BookEntity savedBookEntity = bookService.createUpdateBook(isbn,
bookEntity);
    // 4. Converti l'entità salvata in DTO
    BookDto savedUpdatedBookDto = bookMapper.mapTo(savedBookEntity);

    // 5. Determina lo status code RESTful corretto
    if(bookExists){
        // 200 per aggiornamento
        return new ResponseEntity(savedUpdatedBookDto, HttpStatus.OK);
    } else {
        // 201 per creazione
        return new ResponseEntity(savedUpdatedBookDto, HttpStatus.CREATED);
    }
}
```

4. Verifica Manuale con Postman

Nel video viene mostrato il comportamento dinamico dell'endpoint:

- **Scenario di Creazione:** Si invia una richiesta **PUT** a **/books/978-1-2345-6789-2** con il titolo *"The Shadow in the Basement"*. Poiché l'ISBN non è presente nel database, il server risponde con **201 Created**.
- **Scenario di Aggiornamento:** Mantenendo lo stesso URL con l'ISBN finale **2**, si modifica il titolo nel corpo JSON in *"The Shadow in the Ballroom"*. Inviando nuovamente la richiesta **PUT**, l'applicazione rileva la presenza del record, lo aggiorna e restituisce correttamente lo status **200 OK** con il payload aggiornato.

5. Test di Integrazione Automatizzati con MockMvc

All'interno della classe **BookControllerIntegrationTests**, vengono implementati i test per garantire la stabilità di questa logica di aggiornamento.

A. Test di verifica dello status 200 OK

Questo test assicura che, effettuando un aggiornamento su un libro già esistente nel database, l'endpoint risponda con lo stato 200 OK (e non 201).

```
@Test
public void testThatUpdateBookReturnsHttpStatus200Ok() throws Exception {
    // Crea e salva un libro di test nel database isolato
    BookEntity testBookEntityA = TestDataUtil.createTestBookEntityA(null);
    BookEntity savedBookEntity = bookService.createUpdateBook(
        testBookEntityA.getIsbn(), testBookEntityA
    );

    // Prepara il DTO per la modifica (associando lo stesso ISBN)
    BookDto testBookA = TestDataUtil.createTestBookDtoA(null);
    testBookA.setIsbn(savedBookEntity.getIsbn()); // Allinea l'ISBN
    String bookJson = objectMapper.writeValueAsString(testBookA);

    // Esegue la chiamata PUT e si aspetta 200 OK
    mockMvc.perform(
        MockMvcRequestBuilders.put("/books/" +
savedBookEntity.getIsbn())
            .contentType(MediaType.APPLICATION_JSON)
            .content(bookJson)
        ).andExpect(
            MockMvcResultMatchers.status().isOk()
        );
}
```

B. Test di verifica dell'aggiornamento effettivo nel Body JSON

Questo test verifica che l'aggiornamento modifichi realmente il valore dell'attributo nel database (ad esempio, cambiando il titolo in "updated") e che il JSON di risposta mostri il nuovo valore.

(Nota riguardo le stringhe: è estremamente consigliato se non fondamentale non usare stringhe hardcoded nei test, ma di preferire l'uso delle variabili dinamiche dei DTO e delle entità salvate per evitare falsi positivi).

```
@Test
public void testThatUpdateBookReturnsUpdatedBook() throws Exception {
    // Salva il libro originale nel database di test isolato
    BookEntity testBookEntityA = TestDataUtil.createTestBookEntityA(null);
    BookEntity savedBookEntity = bookService.createUpdateBook(
        testBookEntityA.getIsbn(), testBookEntityA
    );

    // Prepara il DTO e modifica il titolo in "updated"
    BookDto testBookA = TestDataUtil.createTestBookDtoA(null);
    testBookA.setIsbn(savedBookEntity.getIsbn());
    testBookA.setTitle("UPDATED");
    String bookJson = objectMapper.writeValueAsString(testBookA);

    // Esegue il PUT e asserisce sul JSON restituito via JsonPath
    mockMvc.perform(
        MockMvcRequestBuilders.put("/books/" +
            savedBookEntity.getIsbn() )
            .contentType(MediaType.APPLICATION_JSON)
            .content(bookJson)
    ).andExpect(
        // Verifica che l'ISBN sia rimasto invariato
        MockMvcResultMatchers.jsonPath("$.isbn").value("978-1-2345-6789-0") //
        anche con (savedBookEntity.getIsbn())
    ).andExpect(
        // Verifica che il titolo sia stato aggiornato a "updated"
        MockMvcResultMatchers.jsonPath("$.title").value("UPDATED")
    );
}
```

Al termine di questa sezione, eseguendo un intero ciclo di compilazione e test con il comando Maven `mvn clean verify` da terminale, tutti i test di integrazione del controller passano con successo.

10.11 Author Partial Update Endpoint

In questa sezione affrontiamo la progettazione, l'implementazione e il collaudo dell'endpoint per l'**aggiornamento parziale** (*Partial Update*) di un autore.

A differenza dell'aggiornamento completo (**PUT**), l'aggiornamento parziale consente al client di non inviare l'intero oggetto nel corpo della richiesta. Vengono forniti esclusivamente gli attributi che si desidera modificare (ad esempio, solo il nome), lasciando inalterati tutti gli altri campi già memorizzati nel database.

1. Definizione dell'Endpoint nel Controller (**AuthorController**)

Per implementare questa funzionalità in modo RESTful, l'endpoint viene configurato nel controller sfruttando i seguenti elementi:

- **Mapping HTTP:** Si utilizza l'annotazione **@PatchMapping** (o una configurazione equivalente per il metodo PATCH). Il percorso URL è identico a quello del metodo **PUT** (`/authors/{id}`).
- **Firma del Metodo:** Il metodo viene denominato **partialUpdate**. Accetta l'ID della risorsa tramite **@PathVariable("id")** e un oggetto parziale **@RequestBody AuthorDto authorDto**. La risposta viene racchiusa in una **ResponseEntity<AuthorDto>**.

Flusso logico nel Controller:

1. **Verifica dell'esistenza:** Esattamente come per il metodo **PUT**, il controller verifica se la risorsa esiste richiamando **authorService.exists(id)**. Se l'autore non viene trovato, viene restituito lo status **404 Not Found**.
2. **Mappatura DTO-Entità:** Se la risorsa esiste, il DTO (che contiene solo i campi da modificare) viene convertito in un'entità tramite **authorMapper.mapFrom(authorDto)**.
3. **Invocazione del Servizio:** Viene chiamato il metodo **partialUpdate(id, authorEntity)** dello strato di servizio, delegando ad esso la logica di business di aggiornamento e persistenza.
4. **Riconversione e Risposta:** L'entità aggiornata restituita dal servizio viene riconvertita in DTO tramite **authorMapper.mapTo** e restituita al client con lo status **200 OK**.

```

@PatchMapping(path = "/authors/{id}")
public ResponseEntity<AuthorDto> partialUpdate(
    @PathVariable("id") Long id,
    @RequestBody AuthorDto authorDto
) {
    // Verifica se l'autore esiste a DB
    if(!authorService.exists(id)) {
        return new ResponseEntity<>(HttpStatus.NOT_FOUND); // 404
    }

    // Converta il DTO in Entità
    AuthorEntity authorEntity = authorMapper.mapFrom(authorDto);
    // Eseguire l'aggiornamento parziale tramite il servizio
    AuthorEntity updatedAuthor = authorService.partialUpdate(id,
authorEntity);
    // Converta il risultato in DTO e restituisce 200 OK
    return new ResponseEntity<>(
        authorMapper.mapTo(updatedAuthor),
        HttpStatus.OK);
}
}

```

2. Implementazione nel Service Layer (**AuthService**)

Il Service Layer contiene la vera logica di business dell'aggiornamento parziale. Per evitare l'esposizione del database a modifiche indesiderate o non allineate, l'ID viene gestito direttamente all'interno del servizio.

A. Definizione del metodo nell'interfaccia **AuthService**

```

AuthorEntity partialUpdate(Long id, AuthorEntity authorEntity);

```

B. Implementazione in **AuthServiceImpl**

La logica richiede di recuperare prima il record originale dal database, ispezionare quali campi sono stati effettivamente inviati dal client (ovvero quelli non nulli) e sovrascrivere solo quelli sul record originale.

Per fare questo in modo elegante, si utilizza la classe **Optional** e il metodo **ofNullable**:

```

@Override
public AuthorEntity partialUpdate(Long id, AuthorEntity authorEntity) {
    // Assicura l'allineamento dell'ID sull'entità passata
    authorEntity.setId(id);

    // Recupera l'autore esistente dal database
    return authorRepository.findById(id).map(existingAuthor -> {
        // Se il campo "name" dell'entità in ingresso non è null, lo
        // aggiorna sul record esistente
        Optional.ofNullable(authorEntity.getName()).ifPresent(existingAuthor::setName);

        // Se il campo "age" dell'entità in ingresso non è null, lo
        // aggiorna sul record esistente
        Optional.ofNullable(authorEntity.getAge()).ifPresent(existingAuthor::setAge);

        // Salva l'entità esistente con le sole modifiche apportate
        return authorRepository.save(existingAuthor);
    }).orElseThrow(() -> new RuntimeException("Author does not exist")); //
    // Misura di protezione di fallback
}

```

- **Optional.ofNullable(...).ifPresent(...)**: Consente di verificare se una proprietà dell'oggetto inviato è valorizzata. Se lo è, viene eseguita l'espressione lambda che aggiorna il rispettivo setter sul record originale (**existingAuthor**). Se il campo non è presente nel JSON, il valore esistente nel database non viene toccato.
- **.orElseThrow(...)**: Sebbene il controller verifichi preventivamente l'esistenza della risorsa, lo strato di servizio implementa comunque un blocco di sicurezza che lancia una **RuntimeException** qualora la risorsa dovesse risultare mancante durante la transazione.

3. Verifica Manuale con Postman

Il video mostra l'efficacia del metodo **PATCH** simulando una richiesta su un autore esistente (ad esempio, un autore precedentemente modificato in "Maria Montgomery" con ID **1**):

- **Configurazione della richiesta**: Si imposta il metodo HTTP su **PATCH** puntando a <http://localhost:8080/authors/1>.

Payload JSON (Body): Si invia solo il campo che si intende aggiornare (senza ID e senza l'età):

```

{
    "name": "Arya"
}

```

- **Risultato**: Il server restituisce lo status code **200 OK** e l'oggetto completo, in cui l'ID **1** e l'età (es. **80**) sono rimasti inalterati, mentre il nome è stato correttamente

parzializzato e aggiornato in "Arya Montgomery". Il video mostra inoltre come sia possibile aggiornare contemporaneamente più campi (es. inviando nome e l'età impostata a 79) mantenendo lo stesso identico comportamento di successo.

4. Test di Integrazione Automatizzati con MockMvc

All'interno della classe `AuthorControllerIntegrationTests`, vengono implementati due test di integrazione per verificare sistematicamente l'endpoint `PATCH`.

A. Test di verifica dello status HTTP 200 OK

Questo test assicura che una richiesta `PATCH` inviata a un autore esistente risponda con lo stato di successo `200 OK`.

```
@Test
public void testThatPartialUpdateExistingAuthorReturnsHttpStatus200Ok()
throws Exception {
    // Crea e salva un autore di prova nel database di test isolato
    AuthorEntity testAuthorEntityA =
    TestDataUtil.createTestAuthorEntityA();
    AuthorEntity savedAuthor = authorService.save(testAuthorEntityA);

    // Prepara il DTO modificando solo il nome
    AuthorDto testAuthorDtoA = TestDataUtil.createTestAuthorDtoA();
    testAuthorDtoA.setName("UPDATED");
    String authorDtoJson = objectMapper.writeValueAsString(testAuthorDtoA);

    // Esegue la chiamata PATCH e si aspetta status 200 OK
    mockMvc.perform(
        MockMvcRequestBuilders.patch("/authors/" + savedAuthor.getId())
            .contentType(MediaType.APPLICATION_JSON)
            .content(authorDtoJson)
    ).andExpect(MockMvcResultMatchers.status().isOk());
}
```

B. Test di verifica della modifica parziale dei dati nel Body JSON

Questo test assicura che solo il valore inviato (il nome "updated") sia effettivamente cambiato nel database, mentre gli altri campi (come l'età) rimangano esattamente conformi a quelli precedentemente memorizzati.

```
@Test
public void testThatPartialUpdateExistingAuthorReturnsUpdatedAuthor()
throws Exception {
    // Crea e salva "Abigail Rose" (età 80) nel DB di test
    AuthorEntity testAuthorEntityA =
TestDataUtil.createTestAuthorEntityA();
    AuthorEntity savedAuthor = authService.save(testAuthorEntityA);

    // Prepara il DTO parziale impostando il nome su "updated"
    AuthorDto testAuthorDtoA = TestDataUtil.createTestAuthorDtoA();
    // Esegue il PATCH e convalida le modifiche tramite JsonPath
    testAuthorDtoA.setName("UPDATED");
    String authorDtoJson = objectMapper.writeValueAsString(testAuthorDtoA);

    mockMvc.perform(
        MockMvcRequestBuilders.patch("/authors/" + savedAuthor.getId())
            .contentType(MediaType.APPLICATION_JSON)
            .content(authorDtoJson)
    ).andExpect(
        // Verifica che l'ID corrisponda a quello originale salvato
MockMvcResultMatchers.jsonPath("$.id").value(savedAuthor.getId())
    ).andExpect(
        // Verifica che il nome sia stato effettivamente aggiornato a
"updated"
        MockMvcResultMatchers.jsonPath("$.name").value("UPDATED")
    ).andExpect(
        // Verifica che l'età NON sia cambiata e corrisponda ancora a
80
MockMvcResultMatchers.jsonPath("$.age").value(testAuthorDtoA.getAge())
    );
}
```

Eseguendo i test, l'intero flusso di aggiornamento parziale per gli autori si conclude con successo, validando la flessibilità del Presentation e del Service Layer dell'applicazione.

10.12 Book Partial Update Endpoint

In questa sezione viene analizzata e implementata la logica per l'**aggiornamento parziale** (*Partial Update*) della risorsa libro.

Analogamente a quanto fatto per l'autore, l'aggiornamento parziale tramite il metodo HTTP **PATCH** permette al client di inviare solo i campi che desidera realmente modificare (ad esempio, solo il titolo), senza l'obbligo di fornire l'intera struttura del libro nel corpo della richiesta.

1. Il Controller: **BookController**

Nel controller dei libri, l'endpoint viene configurato per intercettare le richieste PATCH sul medesimo percorso dell'URL usato per la creazione e l'aggiornamento completo (`/books/{isbn}`):

- **Mapping HTTP:** Si applica l'annotazione **@PatchMapping** con il percorso `/books/{isbn}`.
- **Firma del Metodo:** Il metodo `partialUpdateBook` accetta l'ISBN estratto dall'URL tramite **@PathVariable("isbn")** e l'oggetto parziale deserializzato come **@RequestBody BookDto bookDto**.
- **Flusso di Controllo:**
 1. Il DTO viene convertito in un'entità JPA provvisoria tramite `bookMapper.mapFrom(bookDto)`.
 2. Si verifica l'effettiva presenza del libro nel database con `bookService.exists(isbn)`. Se la risorsa non esiste, l'API restituisce immediatamente lo status **404 Not Found**.
 3. Se il libro è presente, si delega la logica di unione dati allo strato di servizio invocando `bookService.partialUpdate(isbn, bookEntity)`.
 4. L'entità aggiornata restituita dal servizio viene riconvertita in DTO tramite `bookMapper.mapTo` e rispedita al client con lo status **200 OK**.

```

@PatchMapping(path = "/books/{isbn}")
public ResponseEntity<BookDto> partialUpdateBook(
    @PathVariable("isbn") String isbn,
    @RequestBody BookDto bookDto
){
    // Controllo esistenza risorsa
    if(!bookService.exists(isbn)){
        return new ResponseEntity<>(HttpStatus.NOT_FOUND); // 404
    }

    // Converti il DTO parziale in Entità
    BookEntity bookEntity = bookMapper.mapFrom(bookDto);
    // Esegui la modifica parziale nel Service Layer
    BookEntity updatedBookEntity = bookService.partialUpdate(isbn,
bookEntity);
    // Riconverti in DTO e restituisci 200 OK
    return new ResponseEntity<>(
        bookMapper.mapTo(updatedBookEntity),
        HttpStatus.OK);
}

```

2. Lo Strato di Servizio: **BookServiceImpl**

La logica di business definisce come sovrascrivere selettivamente i dati nel database.

A. Definizione nell'interfaccia **BookService**

```
BookEntity partialUpdate(String isbn, BookEntity bookEntity);
```

B. Implementazione in **BookServiceImpl**

Il servizio deve prima bloccare l'ISBN sul valore sicuro estratto dall'URL, recuperare il record originale dal database, ispezionare quali campi dell'entità in ingresso non sono nulli e aggiornare esclusivamente quelli sul record originale:

```

@Override
public BookEntity partialUpdate(String isbn, BookEntity bookEntity) {
    // Forza l'ISBN corretto sull'entità in ingresso
    bookEntity.setIsbn(isbn);

    // Cerca il libro esistente nel database
    return bookRepository.findById(isbn).map(existingBook -> {
        // Se il client ha inviato un titolo non nullo, lo aggiorna
        Optional.ofNullable(bookEntity.getTitle()).ifPresent(existingBook::setTitle);
        // Nota: non supportiamo l'aggiornamento dell'ISBN o dell'autore
        // nidificato in questa fase
        // Salva l'entità originaria con le sole modifiche apportate
        return bookRepository.save(existingBook);
    }).orElseThrow(() -> new RuntimeException("Book does not exist")); //
    Blocco di sicurezza fallback
}

```

- **Isolamento delle proprietà:** L'implementazione dell'aggiornamento parziale per la risorsa **Book** viene volutamente circoscritta al solo attributo **title**, impedendo la modifica dell'ISBN o dell'autore nidificato.

3. Verifica Manuale con Postman

Nel video si convalida lo sviluppo eseguendo una chiamata a server attivo:

- **Stato Iniziale:** Nel database è presente il libro *"The Shadow in the Attic"*.

Richiesta PATCH: Si invia una richiesta **PATCH** all'URL **/books/978-1-2345-6789-0** con un corpo JSON contenente solo l'attributo modificato:

```

{
  "title": "The Shadow in the Basement",
}

```

- **Risultato:** Il server esegue l'operazione restituendo lo status code **200 OK** e l'oggetto libro completo, in cui il codice ISBN originale è rimasto inalterato, mentre il titolo è stato correttamente parzializzato e modificato in *"The Shadow in the Basement"*.

4. Test di Integrazione Automatizzati con MockMvc

All'interno della classe **BookControllerIntegrationTests**, vengono scritti due test per blindare il comportamento dell'endpoint PATCH.

A. Test di verifica dello status HTTP 200 OK

Questo test garantisce che, modificando un libro esistente, il server risponda con successo con codice 200.

```
@Test
public void testThatPartialUpdateBookReturnsHttpStatus200Ok() throws
Exception {
    // Crea e salva un libro di test nel database isolato
    BookEntity testBookEntityA = TestDataUtil.createTestBookEntityA(null);
    bookService.createUpdateBook(testBookEntityA.getIsbn(),
testBookEntityA);

    // Prepara il DTO modificando solo il titolo
    BookDto testBookA = TestDataUtil.createTestBookDtoA(null);
    testBookA.setTitle("UPDATED");
    String bookJson = objectMapper.writeValueAsString(testBookA);

    // Esegue la chiamata PATCH e si aspetta 200 OK
    mockMvc.perform(
        MockMvcRequestBuilders.patch("/books/" +
testBookEntityA.getIsbn())
            .contentType(MediaType.APPLICATION_JSON)
            .content(bookJson)
    ).andExpect(
        MockMvcResultMatchers.status().isOk()
    );
}
```

B. Test di verifica dell'aggiornamento effettivo via **JsonPath**

Questo test assicura che il payload di risposta rifletta esattamente l'aggiornamento parziale eseguito.

```
@Test
public void testThatPartialUpdateBookReturnsUpdatedBook() throws Exception
{
    // Salva il libro originale nel DB
    BookEntity testBookEntityA = TestDataUtil.createTestBookEntityA(null);
    bookService.createUpdateBook(testBookEntityA.getIsbn(),
testBookEntityA);

    // Prepara il DTO con il titolo "updated"
    BookDto testBookA = TestDataUtil.createTestBookDtoA(null);
    testBookA.setTitle("UPDATED");
    String bookJson = objectMapper.writeValueAsString(testBookA);

    // Esegue la chiamata PATCH e fa asserzioni sul corpo JSON della
risposta
    mockMvc.perform(
        MockMvcRequestBuilders.patch("/books/" +
testBookEntityA.getIsbn())
            .contentType(MediaType.APPLICATION_JSON)
            .content(bookJson)
        ).andExpect(
            // Verifica che l'ISBN non sia mutato
MockMvcResultMatchers.jsonPath("$.isbn").value(testBookEntityA.getIsbn())
        ).andExpect(
            // Verifica che il titolo sia stato aggiornato con successo a
"updated"
            MockMvcResultMatchers.jsonPath("$.title").value("UPDATED")
        );
}
```

Con l'esecuzione di questa suite di test, la logica di aggiornamento parziale per la risorsa libri viene completamente validata.

10.13 Author Delete Endpoint

In questa sezione viene completato il ciclo delle operazioni CRUD per la risorsa "Autore" implementando la lettera **"D" (Delete)**. L'obiettivo è realizzare un endpoint RESTful che consenta di eliminare in modo definitivo un autore dal database utilizzando il suo identificativo unico (ID).

1. Definizione dell'Endpoint nel Controller (**AuthorController**)

Per implementare l'eliminazione in conformità con lo stile RESTful, l'endpoint viene aggiunto all'interno di **AuthorController** seguendo questi passaggi tecnici:

- **Mapping HTTP:** Si utilizza l'annotazione **@DeleteMapping**. Il percorso URL segue la convenzione e include l'ID della risorsa come variabile di percorso: **path = "/authors/{id}"**.
- **Firma del Metodo:** Il metodo viene denominato **deleteAuthor**. Accetta l'ID estratto dall'URL tramite l'annotazione **@PathVariable("id")** (mappato come **Long id**).
- **Nessun Corpo di Risposta:** Poiché l'eliminazione rimuove la risorsa, l'endpoint non deve restituire alcun corpo JSON. Per questo motivo, il metodo restituisce un oggetto **ResponseEntity** generico senza specificare un DTO.
- **Codice di Stato HTTP:** In caso di successo, l'API restituisce lo status code standard REST per le risposte prive di corpo: **204 No Content**.

```
@DeleteMapping(path = "/authors/{id}")
public ResponseEntity deleteAuthor(@PathVariable("id") Long id) {
    // Invocazione del servizio per l'eliminazione
    authService.delete(id);
    // Ritorno dello status code 204 No Content
    return new ResponseEntity(HttpStatus.NO_CONTENT);
}
```

2. Implementazione nel Service Layer e Repository

Per supportare l'operazione del controller, si implementa il metodo corrispondente all'interno dello strato di servizio:

A. Definizione del metodo nell'interfaccia **AuthService**

Il metodo non deve restituire alcun valore (**void**) e accetta l'ID dell'autore da rimuovere:

```
void delete(Long id);
```

B. Implementazione in **AuthServiceImpl**

Grazie all'uso di Spring Data JPA, l'implementazione concreta del metodo è estremamente semplice e non richiede la scrittura di query SQL manuali. Si esegue una chiamata pass-through sfruttando il metodo nativo **deleteById** fornito dall'interfaccia **CrudRepository**:

```
@Override
public void delete(Long id) {
    authorRepository.deleteById(id);
}
```

3. Verifica Manuale con Postman

Nel video viene mostrato come verificare manualmente il corretto funzionamento dell'endpoint attraverso una sequenza logica di tre chiamate:

1. **Verifica Preliminare (GET)**: Si effettua una richiesta **GET** a **http://localhost:8080/authors/1** per accertarsi che l'autore (es. Maria Montgomery) sia effettivamente presente nel database (restituendo **200 OK**).
 2. **Eliminazione (DELETE)**: Si invia una richiesta utilizzando il metodo HTTP **DELETE** verso lo stesso indirizzo (**http://localhost:8080/authors/1**). Il server elabora la richiesta e risponde correttamente con lo status **204 No Content**.
 3. **Verifica Post-Eliminazione (GET)**: Effettuando nuovamente una **GET** sullo stesso ID, il server risponde con lo status code **404 Not Found**, confermando che il record è stato rimosso con successo dal database.
-

4. Test di Integrazione Automatizzati con MockMvc

All'interno della classe **AuthorControllerIntegrationTests**, vengono aggiunti due test di integrazione per blindare il comportamento dell'endpoint.

A. Test 1: Eliminazione di un autore non esistente

Nello standard REST, una richiesta di eliminazione viene considerata di successo anche se la risorsa non esiste già (poiché lo stato finale desiderato, ovvero la non esistenza della risorsa nel sistema, è comunque soddisfatto). Pertanto, il test verifica che, inviando una richiesta **DELETE** con un ID fittizio (es. **99**), l'API risponda comunque con **204 No Content**.

```
@Test
public void testThatDeleteNonExistingAuthorReturnsHttpStatus204NoContent()
throws Exception {
    mockMvc.perform(
        MockMvcRequestBuilders.delete("/authors/999") // ID inesistente
        nel database di test isolato
        .contentType(MediaType.APPLICATION_JSON)
    ).andExpect(MockMvcResultMatchers.status().isNoContent()); // Verifica
dello status 204
}
```

B. Test 2: Eliminazione di un autore realmente esistente

Questo test verifica che l'autore presente nel database venga eliminato con successo.

1. **Preparazione:** Si crea e salva un autore di test ("Abigail Rose") nel database isolato tramite lo strato di servizio (**authorService.save**).
2. **Esecuzione:** Si effettua la chiamata **DELETE** simulata a **/authors/{id}** usando l'ID dell'autore appena salvato, assicurandosi di ricevere uno status **204 No Content**.

```

@Test
public void testThatDeleteAuthorReturnsHttpStatus204ForExistingAuthor()
throws Exception {
    // Crea e salva un autore di prova
    AuthorEntity testAuthorEntityA =
TestDataUtil.createTestAuthorEntityA();
    AuthorEntity savedAuthor = authService.save(testAuthorEntityA);

    // Esegue l'eliminazione tramite chiamata DELETE
    mockMvc.perform(
        MockMvcRequestBuilders.delete("/authors/" +
savedAuthor.getId())
        .contentType(MediaType.APPLICATION_JSON)
    ).andExpect(MockMvcResultMatchers.status().isNoContent()); // Verifica
dello status 204
}

```

La corretta esecuzione di questi test assicura che l'endpoint di eliminazione degli autori sia solido, sicuro ed esente da problemi di regressione.

10.14 Book Delete Endpoint

Quindi procediamo con l'implementazione e il collaudo dell'endpoint per l'**eliminazione definitiva di un libro** utilizzando la sua chiave primaria naturale, ovvero il codice **ISBN**. Trattandosi di una risorsa con un identificatore lineare, il docente la definisce come una delle procedure più rapide del ciclo CRUD, ma la sua integrazione segue meticolosamente i medesimi standard RESTful applicati in precedenza per gli autori.

1. Definizione dell'Endpoint nel Controller (**BookController**)

Per gestire la rimozione di un libro, l'endpoint viene inserito in **BookController** configurando i seguenti elementi tecnici:

- **Mapping HTTP:** Si applica l'annotazione **@DeleteMapping** associata al percorso **/books/{isbn}**. Il parametro dinamico **{isbn}** inserito nell'URL identifica in modo univoco il libro da rimuovere.
- **Firma del Metodo:** Il metodo viene denominato **deleteBook**. Riceve il parametro dinamico dall'URL tramite l'annotazione **@PathVariable("isbn")** (mappato come **String isbn**).
- **Nessun Corpo di Risposta:** Seguendo le convenzioni REST, l'eliminazione di una risorsa non deve restituire alcun payload JSON. Per questo motivo, il metodo restituisce un oggetto **ResponseEntity** generico senza specificare un DTO.
- **Codice di Stato HTTP:** In caso di successo, l'API restituisce lo status code standard REST per le risposte prive di corpo: **204 No Content**.

```
@DeleteMapping(path = "/books/{isbn}")
public ResponseEntity deleteBook(@PathVariable("isbn") String isbn) {
    // Invocazione del servizio per l'eliminazione
    bookService.delete(isbn);
    // Ritorno dello status code 204 No Content
    return new ResponseEntity(HttpStatus.NO_CONTENT);
}
```

2. Implementazione nel Service Layer e Repository

Per supportare l'azione del controller, si definisce il metodo corrispondente all'interno dello strato di servizio:

A. Definizione del metodo nell'interfaccia **BookService**

Il metodo non deve restituire alcun valore (**void**) e accetta l'ISBN del libro da rimuovere:

```
void delete(String isbn);
```

B. Implementazione in **BookServiceImpl**

Grazie a Spring Data JPA, l'implementazione concreta non richiede la scrittura di query SQL manuali. Si esegue una semplice chiamata pass-through sfruttando il metodo nativo **deleteById** fornito dall'interfaccia **CrudRepository** (poiché per l'entità **BookEntity** la chiave primaria è proprio la stringa ISBN):

```
@Override
public void delete(String isbn) {
    bookRepository.deleteById(isbn);
}
```

3. Verifica Manuale con Postman

Nel video viene mostrato come validare il funzionamento dell'endpoint attraverso una sequenza di tre chiamate logiche in Postman:

1. **Lettura Preliminare (GET)**: Si invia una richiesta **GET** a `http://localhost:8080/books/{isbn}` per verificare che il libro (es. *"The Shadow in the Basement"*) sia effettivamente presente nel database (ricevendo **200 OK** e il JSON del libro).
 2. **Eliminazione (DELETE)**: Si invia una richiesta HTTP **DELETE** verso lo stesso indirizzo. Il server elabora la richiesta e risponde correttamente con lo status **204 No Content**.
 3. **Verifica Post-Eliminazione (GET)**: Effettuando nuovamente una **GET** sullo stesso ISBN, il server risponde con lo status code **404 Not Found**, a riprova del fatto che il record è stato cancellato con successo dal database.
-

4. Test di Integrazione Automatizzati con MockMvc

All'interno della classe `BookControllerIntegrationTests`, vengono implementati due test di integrazione per blindare il comportamento dell'endpoint ed evitare regressioni.

A. Test 1: Eliminazione di un libro non esistente

Nello standard REST, una richiesta di eliminazione viene considerata di successo anche se la risorsa non esiste già (poiché lo stato finale desiderato, ovvero l'assenza della risorsa nel sistema, è soddisfatto). Pertanto, il test verifica che, inviando una richiesta `DELETE` con un ISBN fittizio, l'API risponda comunque con **204 No Content**.

(Nota di Debugging nei Test: In fase di configurazione del builder di MockMvc, occorre prestare attenzione ad associare il metodo HTTP corretto (`patch()` anziché `delete()`), causando un errore `400 Bad Request`. Una volta corretto il metodo in `delete()`, il test passa correttamente a verde).

```
@Test
public void testThatDeleteNonExistingBookReturnsHttpStatus204NoContent()
throws Exception {
    mockMvc.perform(
        // Utilizziamo un ISBN fittizio non presente nel database di
test isolato
        MockMvcRequestBuilders.delete("/books/kjsbdfjdfsk")
            .contentType(MediaType.APPLICATION_JSON)
    ).andExpect(MockMvcResultMatchers.status().isNoContent()); // Verifica
dello status 204
}
```

B. Test 2: Eliminazione di un libro realmente esistente

Questo test verifica l'effettiva rimozione di un libro registrato:

1. **Preparazione:** Si crea e salva un libro di test ("Abigail Rose") nel database isolato tramite lo strato di servizio (`bookService.createUpdateBook`).
2. **Esecuzione:** Si effettua la chiamata `DELETE` simulata a `/books/{isbn}` usando l'ID dell'autore appena salvato, assicurandosi di ricevere uno status **204 No Content**.

```
@Test
public void testThatDeleteExistingBookReturnsHttpStatus204NoContent()
throws Exception {
    // Crea e salva un libro di prova
    BookEntity testBookEntityA = TestDataUtil.createTestBookEntityA(null);
    bookService.createUpdateBook(testBookEntityA.getIsbn(),
testBookEntityA);

    // Esegue l'eliminazione tramite chiamata DELETE
    mockMvc.perform(
        MockMvcRequestBuilders.delete("/books/" +
testBookEntityA.getIsbn())
            .contentType(MediaType.APPLICATION_JSON)
        ).andExpect(MockMvcResultMatchers.status().isNoContent()); // Verifica
dello status 204
}
```

Al termine di questo sviluppo, l'esecuzione del comando Maven globale `mvn clean verify` da terminale restituisce un **BUILD SUCCESS**, provando che tutti i test (unitari e di integrazione) dell'intera applicazione passano con successo.

10.15 Gestione degli Oggetti Nidificati

Fino a questo momento, la progettazione delle operazioni CRUD ha evitato di manipolare oggetti nidificati complessi. Ad esempio, non è ancora stato affrontato lo scenario in cui si desidera creare o aggiornare un libro registrando simultaneamente anche il rispettivo autore all'interno dello stesso payload JSON.

Questo limite non è dovuto a Spring Data JPA (che è perfettamente in grado di gestire le operazioni a cascata sui modelli nidificati), ma è legato a un problema di configurazione di **ModelMapper** all'interno dello strato di mappatura DTO-Entità.

1. Il problema: Autore impostato a **null**

Se provi ad avviare l'applicazione con un database vuoto ed effettui una richiesta **PUT** su `/books/{isbn}` inserendo nel corpo della richiesta i dettagli del libro e, in modo nidificato, i dettagli di un nuovo autore (ad esempio, un autore fittizio di nome *"Fred"* di 14 anni), noterai un comportamento imprevisto:

- **Risultato del salvataggio:** Il server crea correttamente il libro e restituisce uno status **201 Created**.
 - **Il problema del dato mancante:** All'interno del JSON di risposta, l'attributo **author** risulta **null**. Effettuando una richiesta **GET** per verificare gli autori a database, la lista risulterà completamente vuota, confermando che l'autore non è stato persistito.
 - **Perché succede?:** Poiché abbiamo rimosso il file `schema.sql` manuale e lasciato che Hibernate generasse la struttura tabelle tramite `ddl-auto`, i vincoli di chiave esterna non sono così restrittivi da impedire il salvataggio di un libro con un autore nullo. Nonostante **BookDto** referenzi correttamente **AuthorDto**, lo strato di mappatura di ModelMapper ignora silenziosamente la conversione dell'oggetto nidificato perché non possiede le istruzioni adatte per farlo.
-

2. La soluzione: Configurazione della Matching Strategy **LOOSE**

La configurazione di default di ModelMapper non è sufficientemente elastica per risolvere le dipendenze degli oggetti nidificati profondi. Per abilitare questa conversione automatica, è necessario modificare la classe di configurazione **MapperConfig**.

Invece di restituire un'istanza base di **ModelMapper**, dobbiamo configurare l'oggetto applicando una **strategia di corrispondenza lenta (loose matching strategy)**. Questa configurazione istruisce la libreria a scendere in profondità nella gerarchia degli oggetti per associare i DTO nidificati alle rispettive entità.

Codice di configurazione modificato in **MapperConfig:**

```
@Configuration
public class MapperConfig {

    @Bean
    public ModelMapper modelMapper() {
        ModelMapper modelMapper = new ModelMapper();
        // Configura la strategia di matching su LOOSE per supportare gli
        // oggetti nidificati
        modelMapper.getConfiguration().setMatchingStrategy(MatchingStrategies.LOOSE);
        return modelMapper;
    }
}
```

3. Come agisce l'unione tra ModelMapper e Spring Data JPA

Una volta implementata la strategia **LOOSE**, il flusso di creazione ed esecuzione cambia radicalmente:

1. **Conversione completa:** Quando il controller riceve il **BookDto** contenente l'oggetto **AuthorDto**, ModelMapper converte l'intera struttura in un'entità **BookEntity** che racchiude al suo interno l'entità **AuthorEntity**.
 2. **Cascading a database:** L'entità **BookEntity** è stata precedentemente configurata con la proprietà **CascadeType.ALL** sulla relazione **@ManyToOne**.
 3. **Persistenza automatica:** Spring Data JPA riceve la **BookEntity** dal servizio, rileva che l'entità **AuthorEntity** nidificata è nuova e priva di ID, e avvia la sequenza del database per generare un nuovo ID per l'autore. Salva l'autore e, subito dopo, inserisce la riga del libro collegandola all'ID dell'autore appena creato.
-

4. Verifica finale dell'endpoint

Dopo aver riavviato l'applicazione con la nuova configurazione, inviando nuovamente la richiesta **PUT** a `/books/978-1-2345-6789-3` con l'autore nidificato *"Fred"*:

- **Risposta del server:** HTTP Status **201 Created**.

Payload JSON:

```
{
  "isbn": "978-1-2345-6789-3",
  "title": "The Shadow in the Basement",
  "author": {
    "id": 1,
    "name": "Fred",
    "age": 14
  }
}
```

- Effettuando una **GET** su `/authors` e `/books`, l'applicazione restituisce correttamente l'autore registrato con ID **1** e il libro ad esso associato.

Questa configurazione completa definitivamente la gestione delle relazioni all'interno dell'API.

10.16 Paginazione

In questa sezione, viene spiegato come implementare il meccanismo di **paginazione**. Fino ad ora, gli endpoint di lettura multipla (come `/books`) restituivano tutti i record presenti nel database in un'unica soluzione. Sebbene questo approccio funzioni con pochi dati, in un'applicazione reale con enormi quantità di record (fino a miliardi) può causare gravi problemi di consumo di risorse, rallentamenti, lunghi tempi di attesa per l'utente o persino il crash del database e dei server applicativi.

La **paginazione** risolve questo problema limitando il numero di elementi restituiti ad ogni richiesta. Spring Boot integra questo meccanismo in modo nativo e trasparente lungo tutta la catena applicativa, dal database (Spring Data JPA) fino al Presentation Layer (Spring Web).

1. Modifiche al Repository Layer

Per sbloccare le funzionalità di paginazione e ordinamento, è necessario modificare l'interfaccia di persistenza del database.

Estensione di `PagingAndSortingRepository`: Oltre a `CrudRepository`, l'interfaccia `BookRepository` deve estendere anche l'interfaccia `PagingAndSortingRepository`.

```
@Repository
public interface BookRepository extends CrudRepository<BookEntity,
String>,
    PagingAndSortingRepository<BookEntity, String> {
}
```

- **Cosa viene sbloccato:** Questa estensione rende disponibile un metodo sovraccaricato `findAll(Pageable pageable)` che accetta un parametro di tipo `Pageable` e restituisce un oggetto di tipo `Page<T>`.

2. Modifiche al Service Layer

Lo strato di servizio viene aggiornato per gestire il trasferimento del parametro di paginazione dal controller al repository.

A. Definizione del metodo nell'interfaccia `BookService`

Si sovraccarica il metodo `findAll` definendo il nuovo tipo di ritorno `Page` e accettando l'oggetto di configurazione `Pageable`:

```
Page<BookEntity> findAll(Pageable pageable);
```

B. Implementazione in `BookServiceImpl`

L'implementazione concreta effettua una chiamata "pass-through" inoltrando il parametro `pageable` direttamente al repository:

```
@Override
public Page<BookEntity> findAll(Pageable pageable) {
    return bookRepository.findAll(pageable);
}
```

3. Modifiche al Controller Layer (`BookController`)

Nel controller dei libri, l'endpoint di lettura collettiva viene aggiornato per raccogliere i parametri di paginazione inviati dal client e restituire la struttura dati corretta.

- **Iniezione automatica di `Pageable`:** Aggiungendo un parametro di tipo `Pageable` nella firma del metodo del controller, Spring Boot è in grado di intercettare automaticamente i parametri di query (Query Parameters) inviati dal client e istanziare l'oggetto con le impostazioni desiderate.
- **Mappatura da `Page<Entity>` a `Page<Dto>`:** Invece di lavorare su uno Stream e raccogliere i dati in una lista, l'oggetto `Page` di Spring Data JPA espone un comodissimo metodo `.map()` nativo. Questo permette di convertire l'intera pagina di entità in una pagina di DTO preservando tutti i metadati di paginazione.

```
@GetMapping(path = "/books")
public Page<BookDto> listBooks(Pageable pageable) {
    Page<BookEntity> books = bookService.findAll(pageable);
    return books.map(bookMapper::mapTo);
}
```

4. Ispezione della Risposta JSON in Postman

Nel video viene mostrato come cambia radicalmente la struttura di risposta dell'endpoint dopo l'introduzione della paginazione.

Se prima l'API restituiva un semplice array JSON (`[...]`), ora restituisce un **oggetto JSON strutturato** che contiene:

1. **Un nodo "`content`":** Un array JSON che racchiude l'elenco effettivo dei record paginati per la pagina corrente.
2. **Un nodo "`pageable`":** Contiene informazioni tecniche sull'offset e sui criteri di ordinamento applicati.

3. **Metadati di paginazione:** Campi di riepilogo estremamente utili per i client (come le applicazioni front-end), tra cui:
- **"totalPages"**: Il numero totale di pagine disponibili nel database.
 - **"totalElements"**: Il numero complessivo di record memorizzati a DB (ad esempio, 40 libri).
 - **"size"**: Il numero massimo di elementi visualizzabili per singola pagina (il valore predefinito di default impostato da Spring Boot è 20).
 - **"number"**: L'indice della pagina corrente.
-

5. Controllo della Paginazione tramite Query Parameters

Il client può controllare in modo dinamico la quantità di dati da scaricare e quale porzione visualizzare aggiungendo parametri all'URL della richiesta HTTP:

- **size**: Definisce quanti elementi mostrare in ogni pagina.
 - *Esempio*: Inviando una richiesta GET a `/books?size=2`, l'array **"content"** restituirà esclusivamente i primi 2 libri del database, riducendo le dimensioni del payload JSON.
- **page**: Definisce quale specifica pagina visualizzare.
 - *Nota importante*: Il conteggio delle pagine segue un indice a **base zero** (la pagina 0 rappresenta la prima pagina, la pagina 1 la seconda, e così via).
 - *Esempio*: `/books?size=2&page=1` restituirà il terzo e il quarto libro memorizzati (ovvero la seconda pagina di risultati se suddivisi a coppie di due).
- **Gestione dei limiti (Out of Bounds)**: Se viene richiesta una pagina inesistente o che supera la quantità di record reali (ad esempio, la pagina 2 quando si richiede un blocco di 100 elementi su un totale di soli 40), il server risponde correttamente restituendo un array **"content"** vuoto (`[]`) e lo status **200 OK**, indicando che non ci sono più elementi da visualizzare.
- **Ordinamento (Sorting)**: Anche l'ordinamento è integrato in questo meccanismo; l'oggetto **pageable** supporta parametri aggiuntivi di query (come **sort**) per consentire di ordinare agevolmente i risultati sul database.

L'adozione della paginazione rappresenta un enorme salto di qualità per l'API, garantendo prestazioni ottimali, flessibilità e una gestione scalabile delle risorse di rete e computazionali.

Deployment su AWS LightSail

In questa ultima parte illustriamo il processo completo per **impacchettare l'applicazione Spring Boot in un contenitore Docker** e **rilasciarla sul cloud utilizzando AWS LightSail**.

Questa sezione è divisa in passaggi logici che coprono la containerizzazione, la configurazione dell'infrastruttura di rete e del database sul cloud, il superamento delle problematiche di avvio (*troubleshooting*) e il test finale dell'applicazione.

1. Creazione del Dockerfile

Per poter eseguire l'applicazione all'interno di un contenitore Docker, è necessario definire un file di configurazione chiamato **Dockerfile** nella radice del progetto. Nel video viene proposto un file di configurazione minimale e lineare, strutturato con le seguenti istruzioni:

```
FROM openjdk:17-jdk-alpine
MAINTAINER devtiro.com
COPY target/*.jar app.jar
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Spiegazione tecnica delle istruzioni:

- **FROM openjdk:17**: Specifica l'immagine di base da utilizzare. L'applicazione viene compilata e testata su **Java 17**, pertanto l'ambiente d'esecuzione del contenitore deve disporre della stessa versione di JDK/JRE.
 - **LABEL maintainer="..."**: Definisce i metadati relativi all'autore o all'organizzazione responsabile dell'immagine.
 - **COPY target/*.jar app.jar**: Copia l'archivio compilato Java (il file **.jar**) generato da Maven all'interno del file system del contenitore, rinominandolo semplicemente in **app.jar**.
 - **ENTRYPOINT ["java", "-jar", "/app.jar"]**: Imposta il comando predefinito che verrà eseguito all'avvio del contenitore, indicando alla Java Virtual Machine di eseguire l'applicazione pacchettizzata.
-

2. Generazione del file **.jar** e Build dell'immagine Docker

Prima di procedere alla compilazione dell'immagine Docker, occorre generare l'artefatto fisico sul proprio computer locale.

Generazione dell'artefatto: Si apre il terminale e si esegue il comando Maven per pulire le vecchie build ed eseguire il packaging completo:

```
./mvnw clean package
```

1. Al termine dell'esecuzione (che si conclude con un *Build Success*), Maven deposita l'archivio `.jar` all'interno della cartella `target/`.

Build dell'immagine: Posizionandosi nella directory principale in cui risiede il `Dockerfile`, si compila l'immagine locale di Docker assegnandole un tag identificativo (ad esempio `devtiro/books`):

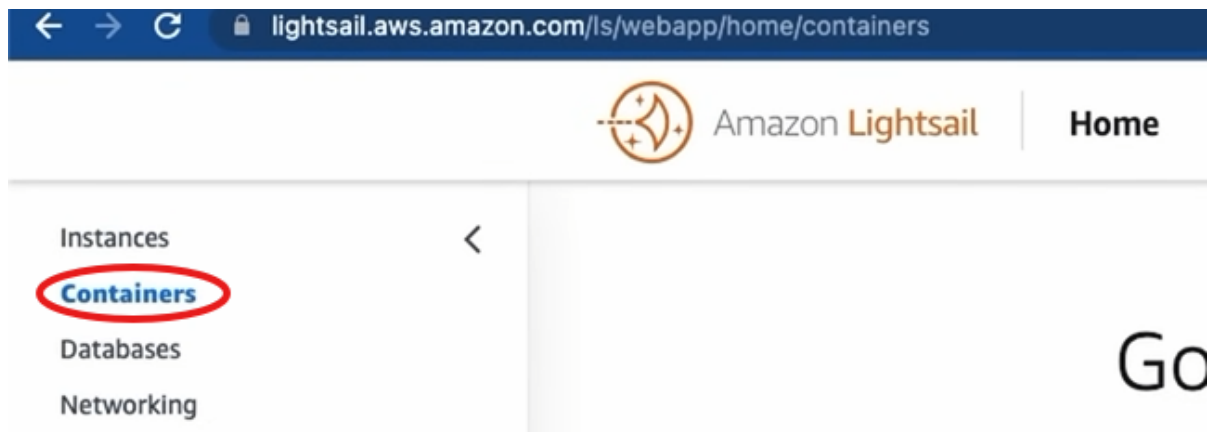
```
docker build -t devtiro/books .
```

2. *Nota:* Il parametro `.` alla fine del comando indica a Docker di cercare il `Dockerfile` e il relativo contesto all'interno della directory corrente.
3. **Verifica:** È possibile verificare la corretta compilazione digitando il comando `docker image ls` per visualizzare l'immagine appena inserita nel catalogo locale con il tag `latest`.

3. Setup del Servizio Container su AWS LightSail

AWS LightSail viene scelto come servizio cloud per la sua semplicità di configurazione e gestione dei contenitori.

Creazione del Servizio: Una volta effettuato l'accesso alla console di AWS LightSail, si accede alla sezione **Containers**:



Si seleziona **Create container service**:



Scelta del piano di calcolo: Per iniziare, si seleziona la configurazione di dimensioni minime (il piano **Nano**) impostando la quantità di nodi su 1:

Container service location



You are creating this container service in **London, all zones** (eu-west-2)

 Change AWS Region

Choose your container service capacity

The power specifies the memory, vCPUs, and base cost of each node in your container service. The scale specifies the number of compute nodes in your container service.

Choose the power

Na Nano \$7 USD per node	Mi Micro \$10 USD per node	Sm Small \$15 USD per node	Md Medium \$40 USD per node	Lg Large \$80 USD per node	Xl Xlarge \$160 USD per node
512 MB RAM	1 GB RAM	1 GB RAM	2 GB RAM	4 GB RAM	8 GB RAM
0.25 vCPUs	0.25 vCPUs	0.5 vCPUs	1 vCPU	2 vCPUs	4 vCPUs

Choose the scale



 Your container service will cost \$7 USD per month.

Your container service includes a data transfer quota of 500 GB per month. Data transfer in excess of the quota will result in an overage charge that varies by AWS Region and starts at \$0.09 USD per GB.

[Learn more about Lightsail pricing](#)

Identificazione del Servizio: Si assegna un nome univoco al servizio di rete (ad esempio `devtiro-books`).

Identify your service

The name of your container service must be unique within each AWS Region in your Lightsail account. It must also be lower-case, and DNS-compliant.

[Learn more about container service names](#)

devtiro-books

The default domain of your container service will be:

devtiro-books.<random-guid>.eu-west-2.cs.amazonlightsail.com

Summary



Your container service will cost \$7 USD per month.

Your container service includes a data transfer quota of 500 GB per month. Data transfer in excess of the quota will result in an overage charge that varies by AWS Region and starts at \$0.09 USD per GB.

[Learn more about Lightsail pricing](#)

Your container service configuration:

✓ **Nano** (512 MB RAM, 0.25 vCPUs) ×1 node



It might take a few minutes for your container service to be available for use.

Create container service

Fatto ciò vi sono una serie di regole da seguire per poter pubblicare un container all'interno di una macchina locale, che sarebbero installare Docker, AWS CLI ed infine il caricamento del nostro container su Lightsail.

Quindi supponendo che le prime due siano state attuate, passiamo direttamente al push su Lightsail.

Upload dell'immagine tramite AWS CLI: Per trasferire l'immagine Docker compilata localmente nel registro privato di AWS LightSail, si utilizza l'interfaccia a riga di comando di AWS (AWS CLI).

Dove:

Inserisci il seguente comando per caricare l'immagine del container dal tuo computer locale sul tuo servizio container.

```
aws lightsail push-container-image --region <Region> --service  
<ContainerServiceName> --label <ContainerImageLabel> --image  
<LocalContainerImageName>:<ImageTag>
```

Nel comando, sostituisci:

- **<Region>** con la regione AWS in cui è stato creato il tuo servizio container.
- **<ContainerServiceName>** con il nome del tuo servizio container.

- **<ContainerImageLabel>** con l'etichetta che desideri assegnare all'immagine del container quando viene archiviata sul tuo servizio container. Specifica un'etichetta descrittiva che puoi utilizzare per tenere traccia delle diverse versioni delle immagini dei container registrate.

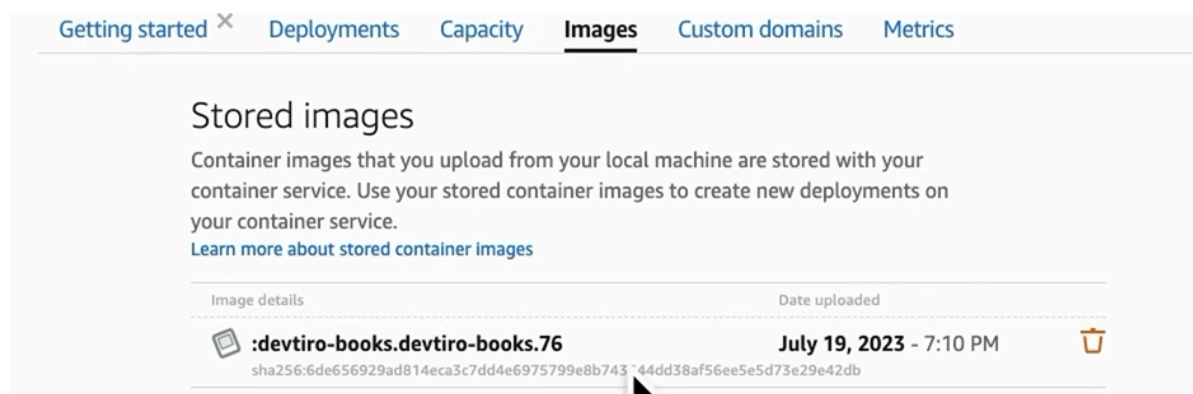
L'etichetta farà parte del nome dell'immagine del container generato dal proprio servizio container. Ad esempio, se il nome del proprio servizio container è **container-service-1**, l'etichetta dell'immagine del container è **mystaticsite** e questa è la prima versione dell'immagine del container che si sta inviando, allora il nome dell'immagine generato dal proprio servizio container sarà **:container-service-1.mystaticsite.1**.

- **<LocalContainerImageName>** con il nome dell'immagine del container che si desidera caricare sul proprio servizio container. Il nome dell'immagine del container è stato ottenuto nel passaggio precedente di questa procedura.
- **<ImageTag>** con il tag dell'immagine del container che si desidera caricare sul proprio servizio container. Il tag dell'immagine del container è stato ottenuto nel passaggio precedente di questa procedura.

Di seguito il **comando AWS CLI** per eseguire il caricamento dell'immagine Docker nel registro privato di LightSail:

```
aws lightsail push-container-image --region eu-west-2 --service devtiro-books --label devtiro-books --image devtiro/books:latest
```

Al termine del caricamento, la console web mostrerà l'immagine registrata nel cloud pronta per il deployment.



4. Configurazione del Database PostgreSQL su AWS

Poiché l'applicazione necessita di un database PostgreSQL reale per memorizzare i dati (configurazione già implementata nel Modulo 8), occorre configurare un'istanza relazionale remota.

Creazione del database: All'interno di AWS LightSail, si crea un database PostgreSQL standard denominato `devtiro-books` (con il nome effettivo del database impostato su `devtiro-books-db`).

Create a database

Database location

You are creating this database in **London, Zone A** (eu-west-2a)
 [Change AWS Region and Availability Zone](#)

Pick your database

MySQL
8.0.33

PostgreSQL
14.8

Choose your database plan

Standard

High Availability

Standard plan databases are created in one Availability Zone only, with no redundancy

Sort by:

Price per month

Memory

Processing

Storage

Transfer

First 3 months free!

\$15

USD

\$30

USD

\$60

USD

\$115

USD

Identify your database

Your Lightsail resources must have unique names.

devtiro-books-db

Recupero dei parametri di connessione: Ad attivazione completata, accedendo alla pagina del nostro database, avremmo davanti questa schermata:



devtiro-books-db

1 GB RAM, 2 vCPUs, 40 GB SSD
PostgreSQL database (14.8)
London, Zone A (eu-west-2a)

Endpoint: **ls-52946c3659e692341430405ed6718d27bde8d6d3.cxdxuas9xps.eu-west-2.rds.amazonaws.com**

Port: **5432**

Stop

Reboot

Status: **Available**

[Connect](#) [Logs & history](#) [Metrics](#) [Networking](#) [Snapshots & restore](#) [Tags](#) [Delete](#)

Connection details

User name and password ?

User name
dbmasteruser

Password (show)

[Change password](#)

Endpoint and port ?

Endpoint
ls-52946c3659e692341430405ed6718d27bde8d6d3.cxdxuas9xps.eu-west-2.rds.amazonaws.com

Port
5432

 **Public mode is disabled.**
Only your Lightsail resources in the same Region as your database can connect to it.

Questi sono parametri essenziali forniti da AWS (Consiglio vivamente di annotare i parametri all'interno di un file di configurazione locale o di ambiente):

- **Endpoint** (l'indirizzo URL del server di database remoto).
- **Username**.
- **Password** di accesso generata.
- Il file in questione (I valori mostrati sono credenziali fittizie a scopo dimostrativo):

```
jdbc:postgresql://ls-52946c3659e692341430405ed6718d27bde8d6d3.cxdxuas9xps.eu-west-2.rds.amazonaws.com:5432/postgres
ls-52946c3659e692341430405ed6718d27bde8d6d3.cxdxuas9xps.eu-west-2.rds.amazonaws.com

SPRING_DATASOURCE_USERNAME
dbmasteruser

SPRING_DATASOURCE_PASSWORD
KF[{5ZPWNUw%z>Pg]:vGH2+rO{G4A;%e
```

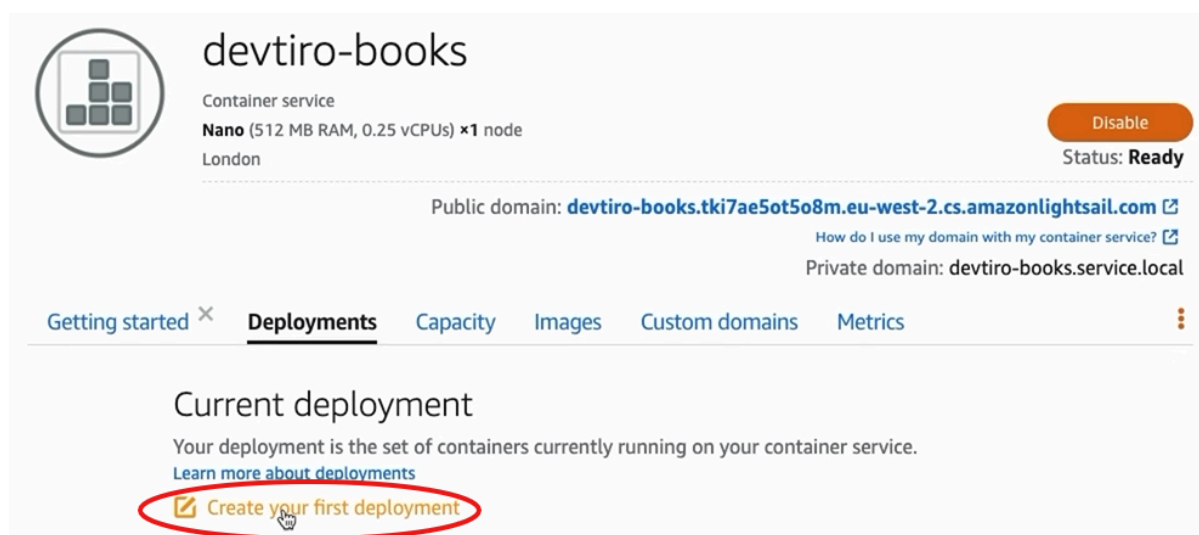
Isolamento e Sicurezza: Per motivi di sicurezza, si raccomanda di mantenere disabilitato l'accesso pubblico all'istanza del database. AWS LightSail consentirà comunque ai servizi container ospitati all'interno della medesima regione di comunicare privatamente tra loro.

5. Configurazione del Deployment e Mapping delle Variabili d'Ambiente

Per far sì che l'applicazione in esecuzione sul contenitore possa comunicare con il database PostgreSQL appena creato, è necessario passare le credenziali di connessione all'avvio.

Come spiegato nel Modulo 4, Spring Boot è in grado di leggere le configurazioni direttamente dalle **variabili d'ambiente** del sistema. Qualsiasi chiave contenuta nel file `application.properties` (come `spring.datasource.url`) viene automaticamente mappata convertendo i caratteri in **maiuscolo** e sostituendo i punti con i caratteri di **underscore** (`_`).

All'interno della schermata di configurazione del deployment del container AWS LightSail:



Viene innanzitutto creato il primo deployment:

 **Create your first deployment**

 Saving this deployment will create a new deployment version

CONTAINERS

Remove X

Container name
Container names must contain only alphanumeric characters and hyphens. A hyphen (-) can separate words but cannot be at the start or end of the name.

Image
Enter the image reference from a public registry, such as DockerHub.

 Choose stored image

Configuration
Optionally specify a command, the environment variables, and the ports to open on your container.
Launch command:
 Add environment variables
 Add open ports

Vengono poi impostati i seguenti parametri:

- **Porte Aperte:** Si configura la porta **8080** (la porta predefinita su cui rimane in ascolto l'applicazione Spring Boot) affinché sia accessibile pubblicamente dall'esterno.
- **Variabili d'ambiente:**
 - **SPRING_DATASOURCE_URL:** Impostata con la stringa di connessione JDBC strutturata nel seguente formato:
`jdbc:postgresql://<ENDPOINT_AWS_DATABASE>:5432/postgres`
(facendo puntare la connessione all'host remoto fornito dalla console AWS).
 - **SPRING_DATASOURCE_USERNAME:** Impostata con l'utente amministrativo del DB remoto.
 - **SPRING_DATASOURCE_PASSWORD:** Impostata con la password di sicurezza annotata in precedenza.

Configuration

Optionally specify a command, the environment variables, and the ports to open on your container.

Launch command:

Environment variables

Key	Value (optional)	
SPRING_DATA	jdbc:postgresql://ls-52946c3659e6923414304	✗
SPRING_DATA	dbmasteruser	✗
SPRING_DATA	KF[{5ZPWNUw%z>Pg]:vGH2+rO{G4A;%e I	✗

+ Add variable

Open ports

Your application code for this container must listen to a port specified here.

Port	Protocol	
8080	HTTP ▼	✗

+ Add port

Ed infine si va a scegliere l'endpoint dell'applicazione per poi andare a salvare e deployare il tutto:

+ Add container entry

i You can have up to 10 containers in a deployment

PUBLIC ENDPOINT

Choose a container in your deployment that you want to make available to the internet as a public endpoint. Make sure to open an HTTP or HTTPS port on the selected container configuration, and then choose it as the port of your public endpoint.

i The container you choose as your public endpoint must respond to traffic on the specified port.

▼

Port

▼
 ?

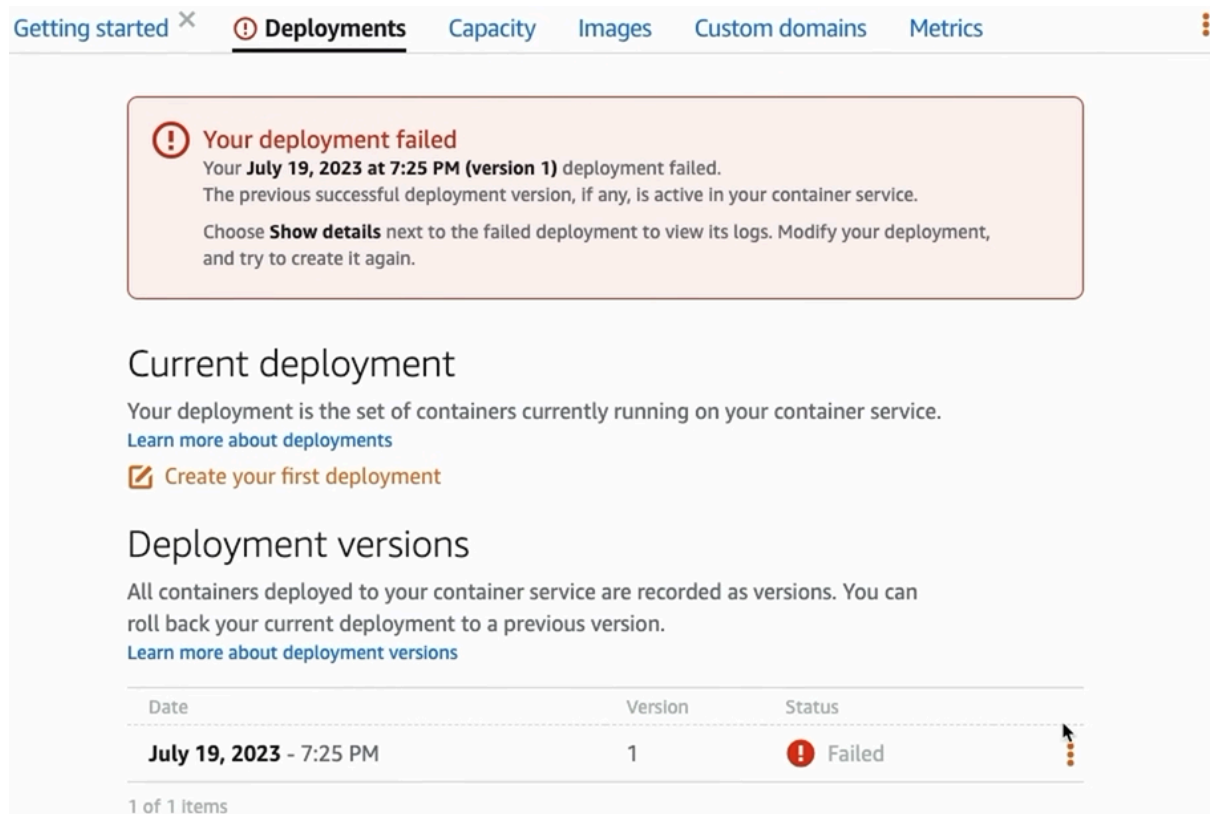
Health check path

🔍 Show all settings

Cancel *🚫* Save and deploy *✅*

6. Gestione dei Problemi di Health Check e Troubleshooting

Durante il primo tentativo di rilascio sul cloud, il deployment potrebbe fallire con un errore di "Health Check non superato".



The screenshot shows the AWS Management Console interface. At the top, there are tabs: "Getting started", "Deployments", "Capacity", "Images", "Custom domains", and "Metrics". The "Deployments" tab is selected. Below the tabs, there is a red alert box with a warning icon and the text: "Your deployment failed. Your July 19, 2023 at 7:25 PM (version 1) deployment failed. The previous successful deployment version, if any, is active in your container service. Choose Show details next to the failed deployment to view its logs. Modify your deployment, and try to create it again." Below the alert box, there is a section titled "Current deployment" with a description: "Your deployment is the set of containers currently running on your container service." and a link "Learn more about deployments". Below this, there is a button "Create your first deployment". Below the button, there is a section titled "Deployment versions" with a description: "All containers deployed to your container service are recorded as versions. You can roll back your current deployment to a previous version." and a link "Learn more about deployment versions". Below the description, there is a table with columns "Date", "Version", and "Status". The table has one row: "July 19, 2023 - 7:25 PM", "1", and "Failed". Below the table, there is a link "Show details" and a button "Roll back".

Date	Version	Status
July 19, 2023 - 7:25 PM	1	Failed

Questo accade perché i controlli automatici di AWS non ricevono una risposta positiva sulla porta 8080 entro i tempi limite stabiliti, contrassegnando l'istanza come non sana (*unhealthy*) e interrompendo l'avvio.

Di fatto controllando all'interno dei log dell'applicazione possiamo intravedere messaggi come:

- [deployment: 1] Took too long
- [deployment:1] Health checks failed: port 8080 is unhealthy

Nel video vengono presentate le soluzioni per risolvere questo specifico blocco infrastrutturale:

1. **Fattore Risorse (Memory Constraint):** Il livello di calcolo minimo "Nano" dispone di risorse hardware (RAM) estremamente limitate, rallentando notevolmente il tempo di bootstrap di Spring Boot. Per ovviare a questo problema, si effettua un upgrade del piano di servizio del container passando alla dimensione **Micro**, che offre prestazioni superiori e tempi di avvio più rapidi.
2. **Rilassamento dei Parametri di Health Check:** Per dare all'applicazione il tempo necessario di avviarsi completamente e connettersi al database prima che AWS

esegua il controllo, si configurano le impostazioni di Health Check in modo più tollerante:

- **Endpoint Path:** Impostato su `/books` (poiché REST risponde con un codice `200 OK` sicuro anche in assenza di dati, a differenza di una radice `/` non mappata).
- **Timeout:** Incrementato a **12 secondi**.
- **Intervallo di controllo:** Impostato a **120 secondi**.
- **Impostazioni di tolleranza complessiva:** Si configurano fino a 120 secondi tra un controllo e l'altro, concedendo fino a 5 tentativi consecutivi prima di dichiarare ufficialmente l'istanza non sana.

CONTAINERS

Public endpoint

> devtiro-books

Image: :devtiro-books.devtiro-books.76

Open log

PUBLIC ENDPOINT

devtiro-books

Port	Protocol
8080	HTTP
Health check path	
/books	
Health check timeout seconds	
12	
Health check interval seconds	
120	
Health check success codes	
200-499	
Health check healthy threshold	
2	
Health check unhealthy threshold	
5	

Less details

Una volta applicate queste modifiche e avviata l'istanza, i log mostreranno la corretta inizializzazione del pool di connessioni Hikari e l'avvio definitivo di Tomcat sulla porta 8080.

7. Test Funzionale dell'API su AWS tramite Postman

A deployment completato, AWS LightSail assegna un dominio pubblico HTTPS sicuro all'applicazione. Per verificare che tutto funzioni esattamente come nell'ambiente di sviluppo locale, si effettuano i seguenti test tramite Postman:

- **Quirk di Rete:** È di fondamentale importanza assicurarsi di utilizzare il protocollo sicuro `https://` all'interno dell'URL di richiesta in Postman, altrimenti la chiamata non verrà instradata correttamente sul dominio pubblico di LightSail.

- **Test di Creazione (PUT):** Si invia una richiesta **PUT** a https://<DOMINIO_AWS>/books/978-1-2345-6789-3 contenente il JSON del libro e i dettagli dell'autore nidificato ("Fred", età 14).
 - *Esito:* Il server risponde con successo restituendo lo status code **201 Created**.
- **Test di Lettura (GET):** Effettuando richieste **GET** agli indirizzi **/authors** e **/books**, l'applicazione sul cloud interroga il database PostgreSQL remoto e restituisce i record appena inseriti nel formato JSON corretto, confermando il perfetto funzionamento dell'intera infrastruttura cloud.